

AI-Enhanced Defence Mechanisms for Mitigating Cyber Attacks on Pilgrimage Applications: A Case Study on Nusuk

Graduation Project, Part-I

Computer Science Department

Jamoum University College

Umm Alqura University

Submitted by

Manar Obaidallah Albeshri ,444002845

Riyof Abdullah Alshaikh, 444000553

Rania Salh Albeshri,444009852

Roud Saad Albshri ,444001485

Project Advisor:

Ebtisaam Abdullah Alharbi

Date submitted

Table of Contents

Chapter 1 ..... **11**

Introduction *11*

1.1 Background and Motivation 11

1.2 Problem Statement 11

1.3 Objectives 12

1.4 Project Scope and Limitations 13

Chapter 2 ..... **15**

Literature Review *15*

2.1 Introduction *15*

2.2 Review of Related Studies *15*

2.3 Attacks on Pilgrimage Applications *16*

2.3.1 Distributed Denial of Service (DDoS) 16

2.3.2 Phishing and Fake-Service Scams 17

2.3.3 API Abuse and Targeted API Attacks 17

2.3.4 Data Breaches 18

2.3.5 Man-in-the-Middle (MitM) and Insecure Network Interception 18

2.3.6 Social Engineering — Targeted Messaging and SMShing/WhatsApp Scams 19

2.3.7 Exploitation of Application Vulnerabilities (SQLi, XSS, RCE, insecure dependencies) 19

2.3.8 Supply-chain / Third-party Provider Attacks 20

2.3.9 Malware Distribution via Fake Updates or Malicious Attachments 20

2.4 Defense Mechanisms Against Application Attacks *21*

2.4.1 Distributed Denial of Service (DDoS) — including API-targeted floods and probing 21

2.4.2 Phishing and Fake-Service Scams (including fake companies, registration/lottery scams) 21

2.4.3 API Abuse and Targeted API Attacks 22

2.4.4 Data Breaches — Database Compromise & Sensitive Data Leaks 22

2.4.5 Man-in-the-Middle (MitM) and Insecure Network Interception 22

2.4.6 Social Engineering, Targeted Messaging, and SMShing/WhatsApp Scams 22

2.4.7 Exploitation of Application Vulnerabilities (SQLi, XSS, RCE, insecure dependencies) 23

2.4.8 Supply-chain / Third-party Provider Attacks 23

2.4.9 Malware Distribution via Fake Updates or Malicious Attachments 23

2.5 Research Gap and Justification *25*

2.6 Feasibility Study *26*

2.7 Summary *26*

Chapter 3 ..... **27**

Methodology **27**

3.1 Research Methodology *27*

3.2 Tools and Technologies / Data Collection Methods *27*

3.2.1 Frontend Development *27*

3.2.2 Backend and Cloud Services *27*

.٣،٢،٣1 Public Intrusion Detection Dataset (Primary Data Source) 28

.2 Dataset Structure 28

3.٢،٣.3 Dataset Features 29

4 Data Transformation 29

.5 Use of Data in This Project 30

3.2.4 Cybersecurity Simulation and Testing *32*

3.2.5 Artificial Intelligence and Machine Learning *33*

3.2.6 Version Control and Project Management *33*

3.3 Project Timeline *35*

3.4 Risk Management *35*

1. Risk Identification 35

2. Risk Analysis 35

3. Risk Mitigation Strategies 36

4. Continuous Monitoring 36

3.5 Validation and Reliability *37*

1. Model Validation Using Unseen Data 37

2. Evaluation Metrics 37

3. Real-Time Detection and Filtering Validation 38

3.6 Summary *39*

Chapter 4 ..... **40**

Project Requirements & Analysis **40**

4.1 Stakeholder Identification *40*

4.2 Methods of Requirements Gathering / Data Needs and Sources *41*

4.2.1 Data Needs and Sources Summary 41

- **Public Intrusion Detection Dataset:** 41
- **System Interaction Data (Prototype-Level):** 41
- **Log Data for Testing the System Interface:** 41
- **Detection Output Data:** 41

4.3 Functional Requirements *42*

4.3.1 Functional Requirements for System Administrator / Ministry 42

4.3.2 Functional Requirements for End Users (Pilgrims / Residents) 42

4.3.3 Functional Requirements for Developers / Research Team 43

4.4 Non-Functional Requirements *43*

4.4.1 Performance & Scalability 43

4.4.2 Availability & Reliability 44

4.4.3 Security & Privacy 44

4.4.4 Maintainability & Modifiability 45

4.4.5 Safety & Ethical Considerations 45

4.5 Constraints *46*

1. **Resource Constraints:** 46
2. **Ethical and Legal Constraints:** 46
3. **Time Constraints:** 46

4. **Resource Cost Constraints:** 46
5. **Tool and Technology Compatibility:** 46
6. **Data Availability:** 47

4.6 Summary 47

Chapter 5 48

System Design 48

5.1 Overview of Design Approach 48

5.2 Data Flow Analysis 49

5.3 System Architecture 50
1. **Client Layer:** 51
2. **Web Server Layer:** 51
3. **ML Detection Layer:** 51
4. **Filtering & Response Layer:** 51
5. **Web Application Backend:** 51

5.4 Use Case Analysis 51

5.4.1 Actors 52
Admin 52
Pilgrim / User 52
Developer / Research Team 52

5.4.2 Use Cases for Admin 52
1. **Show Alerts:** 52
2. **Manage Users:** 52
3. **Configure Services & Slots**: 52
4. **Approve / Reject Requests:** 52
5. **Manage Bookings:** 52
6. **Audit Logs & Activity:** 52

5.4.3 Use Cases for Pilgrim / User 53
1. **Register:** 53
2. **Login:** 53
3. **Browse Services:** 53
4. **Select Time Slot:** 53
5. **Submit Booking Request:** 53
6. **Upload Documents:** 53
7. **View Booking Status:** 53
8. **Modify / Cancel Booking:** 53

5.4.4 Use Cases for Developer / Research Team 53
1. **Run Attack Simulation:** 53
2. **Configure Attack Parameters:** 53
3. **Add New Attack Scenarios:** 53
4. **Capture Network Traffic Logs (PCAP):** 53
5. **Export Logs & Telemetry:** 53
6. **Monitor Alerts (SIEM):** 53
7. **Label Data / Ground Truth:** 53
8. **Retrain Detection Model:** 53

5.4.5 Use Case Relationships 54
- **Login extends Register:** 54

- **Select Time Slot includes Permits and Haramain Train:** 54
- **Retrain Detection Model includes Label Data:** 54

5.5 System Use Case Diagram 54

5.6 System Use Case Description 57

5.6.1 Pilgrim use case description 57

5.6.2 Admin use case description 61

5.7 Entity-Relationship (ER) Analysis 76

Overall, this ER diagram demonstrates a unified database design that supports both the functional requirements of the pilgrimage application and the AI-based intrusion detection process, enabling efficient data management, auditability, and proactive cyber threat response. 76

5.8 System Class Diagram 77

5.9 System Interaction Diagram 79

This diagram illustrates the system's operational flows, detailing the step-by-step data exchange for both user booking requests and the automated attack detection process. 79

1 Scenario 1: Booking Request Flow Description 79

2 Scenario 2: Attack Detection Flow Description 79

5.10 System Activity Diagram 80

5.11 Front-End / User Interface Design 82

5.11.1 Landing Screen 82

5.11.2 Login Screen 82

5.11.3 Services Home Screen 83

5.11.4 Dashboard 84

5.12 Database / Data Model Design 85

5.12.1 Core Application Tables 85

5.12.2 Attack Simulation Tables 86

5.12.3 Detection and Alert Tables 86

5.12.4 Design Features 86

5.13 Algorithm / Process Design 87

5.13.1 Dataset Preparation 87

5.13.2 Data Preprocessing 88

1. Removing Missing or Invalid Values 88

2. Feature Selection 88

3. Normalization 88

5.13.3 Attack Detection Algorithm 88

Process Flow 88

1. Traffic Capture 88

2. Feature Extraction 88

3. ML Prediction: 88

4. Alert Generation: 88

5.13.4 Feature Extraction Process 89

Extracted Features Include: 89

Process 89

5.13.5 Traffic Filtering and Mitigation Process 89

Process Flow 89

5.13.6 Booking Management Process 90

Process Flow 90

..... *91*

5.13.7 Attack Simulation Process *91*
Process Flow 91

5.13.8 Model Training Process *92*
Process Flow 92

5.14 Summary *92*
5.14.1 Design Overview 92
5.14.2 Key Design Components 92
5.14.3 Design Decisions and Rationale 93
5.14.4 Link to Implementation Stage 93
5.14.5 Implementation Priorities 94

6 Chapter 6 ..... **96**

6.1 Implementation Plan *96*
6.1.1 Phase 1: Environment Setup 96
6.1.2 Phase 2: Data Preparation 96
6.1.3 Phase 3: Model Development 97
6.1.4 Phase 4: Evaluation and Comparison 97
6.1.5 Phase 5: System Development 102

6.2 Deployment or Execution *106*
6.2.1 Operating Characteristics 106
6.2.2 Execution Steps 106

6.3 Summary *106*

7 Chapter 7 ..... **108**

7.1 Results Presentation *108*
7.1.1 Evaluation Metrics 108
7.1.2 Performance Evaluation 108

7.2 Usability Testing / User Feedback *111*
7.2.1 Key Findings: 111
7.2.2 Suggested Improvements: 111

7.3 Comparative Analysis *111*
7.3.1 Comparison with Related Work 111
7.3.2 Key Observations: 111
٧,٣,٣ Discussion of Findings 112

7.4 Challenges Encountered and Lessons Learned *113*
Challenges: 113
Solutions: 113
Lessons Learned: 113

Limitations *113*

7.5 Summary *113*

8 Chapter 8 ..... **114**

8.1 Summary of Work *114*

8.2 Key Findings *115*

8.3 Future Work *116*

8.3.1 Improving Model Performance 116

8.3.1 Real-Time Implementation 116

8.3.2 Expanding Attack Detection 116

8.3.3 Enhancing the User Interface 116

8.3.4 Advanced AI Integration 116

8.4 Final Conclusion *117*

Abstract

This project presents the development of an intelligent cyberattack detection system designed to analyze and identify Denial-of-Service (DoS) and Distributed Denial-of-Service (DDoS) attacks targeting critical digital applications. The project follows a structured experimental methodology that includes cybersecurity data preparation, attack behavior analysis, machine learning model implementation, and performance evaluation using publicly available intrusion detection datasets.

Several machine learning and deep learning algorithms were implemented and compared, including Support Vector Machine (SVM), Random Forest, Logistic Regression, and Long Short-Term Memory (LSTM). The experiments were conducted using the CICDDoS2019 and UNSW-NB15 datasets under different evaluation scenarios to assess detection accuracy, stability, and generalization capability.

The proposed system analyzes network traffic patterns and identifies abnormal behaviors associated with malicious activities. The evaluation results demonstrated strong detection performance across different experimental scenarios, with Random Forest achieving the most stable and consistent overall results.

The project highlights the effectiveness of machine learning techniques in improving cyberattack detection accuracy and supporting the development of more secure and reliable digital systems.

List of Figures

Figure 1 Project Phases 13
Figure 2 Limitations and Constraints 14
Figure 3 Gaps in Literature 25
Figure 4 dataset Structure CICDDoS2019 29
Figure 5 Dataset Structure UNSW NB15 31
Figure 6 TimeLine 35
Figure 7 Risk Identification 37
Figure 8 Data Flow 49
Figure 9 System Architecture 50
Figure 10 User use case diagram 54
Figure 11 admin use case diagram 55
Figure 12 Research Team use case diagram 56
Figure 13 Entity-Relationship (ER) 76
Figure 14 System Class Diagram 77
Figure 15 System Interaction Diagram 79
Figure 16 System Activity Diagram 80
Figure 17 Landing Screen 82
Figure 18 Login Screen 83
Figure 19 Services Home Screen 84
Figure 20 Dashboard 85
Figure 21 Database Schema Diagram 87
Figure 22 Attack Detection Algorithm 89
Figure 23 Traffic Filtering and Response Based on Attack Severity 90
Figure 24 Booking Management Process 90
Figure 25 Booking Approval Process 91
Figure 26 Attack Simulation Process 91
Figure 27 Implementation Priorities 95
Figure 28. Experimental Evaluation Scenarios 98
Figure 29 Accuracy comparison of machine learning models 100
Figure 30 Accuracy comparison of machine learning models using the merged dataset evaluation approach over 100 runs on CICDDoS2019 101
Figure 31 accuracy Comparison of ML Models on UNSW-NB15 Dataset 102
Figure 32 Security Dashboard 105
Figure 33 Security Alert Interface 105
Figure 34 Execution Steps 106
Figure ٣٥ Comparison between UNSW NB15 and CICDDos2019 on Random Fores.... خطأ! الإشارة المرجعية غير معرّفة.
Figure 36 confusion matrices on UNSW-NB15 110

List of Tables

Table 1 Cybersecurity Defences: AI vs. Traditional Methods 24
Table 2 Project Tools and Technologies Overview 34
Table 3 Stakeholder Identification 41
Table 4 Functional Requirements for System Administrator / Ministy 42
Table 5 Functional Requirements for End Users (Pilgrims / Residents) 42
Table 6 Functional Requirements for Developers / Research Team 43
Table 7 Non-Functional Requirements performance and scalability 43
Table 8 Non-Functional Requirements Availability and Reliability 44
Table 9 Non-Functional Requirements Security and Privacy 45
Table 10 Non-Functional Requirements Maintainability and Modifiability 45
Table 11 Non-Functional Requirements Safety & Ethical Considerations 45
Table 12 Pilgrim use case description 1 57
Table 13 Pilgrim use case description2 58
Table 14 Pilgrim use case description3 59
Table 15 Pilgrim use case description4 60
Table 16 Admin use case description 1 61
Table 17 Admin use case description 2 62
Table 18 Admin use case description 3 63
Table 19 Admin use case description 4 64
Table 20 Admin use case description 5 65
Table 21 Admin use case description 6 66
Table 22 Research Team use case description 1 67
Table 23 Research Team use case description 2 68
Table 24 Research Team use case description 3 69
Table 25 Research Team use case description 4 70
Table 26 Research Team use case description 5 71
Table 27 Research Team use case description 6 72
Table 28 Research Team use case description 7 73
Table 29 Research Team use case description 8 74
Table 30 Research Team use case description 9 75
Table 31Performance Comparison on CICDDoS2019 109
Table 32 Performance on UNSW-NB15 Dataset 109

Chapter 1

Introduction

1.1 Background and Motivation

The digital transformation of pilgrimage services has introduced convenient platforms that greatly facilitate pilgrims' access to essential information and services. Among these platforms, the Nusuk application—developed to support Hajj and Umrah procedures—plays a central role in enabling registration, bookings, notifications, and service coordination for millions of users. As reliance on such platforms increases, so does their exposure to cyber threats. High-traffic periods such as the Hajj season amplify risk, as attackers may exploit load spikes and potential vulnerabilities to launch disruptive campaigns (e.g., Distributed Denial of Service — DDoS) or to exfiltrate sensitive user data .

This project is motivated by the urgent need to enhance the cybersecurity resilience of pilgrimage applications by :

(1) creating a reproducible simulation environment that models realistic attack scenarios

(2) Integrating automated detection and mitigation mechanisms driven by artificial intelligence to preserve availability, integrity, and confidentiality during adverse conditions.

1.2 Problem Statement

The increasing digitization of Hajj and Umrah services has elevated the operational importance of platforms like Nusuk. However, existing security countermeasures may not be sufficient against large-scale, sophisticated cyberattacks that target availability and data confidentiality during peak usage. Such incidents can lead to severe consequences:

- Service disruption during critical times, causing operational and safety impacts for pilgrims;
- Compromise of personal and sensitive data, leading to privacy violations and legal liabilities;
- Loss of public trust in digital services provided by authorities, reducing adoption and effectiveness of e-services.

Despite existing best-practice guidelines and tools, there is a gap in practical, repeatable frameworks that enable stakeholders (developers, security teams, and decision-makers) to: (a) evaluate system behavior under representative attack loads, (b) test the effectiveness of layered defenses, including AI-based detection, and (c) prepare validated incident response playbooks. This project addresses that gap by delivering a simulation system and an evidence-based evaluation of defense strategies tailored to the Nusuk-like environment.

1.3 Objectives

The primary objective is to design, implement, and evaluate a simulation-based security framework for Nusuk-like pilgrimage applications that incorporates AI-driven detection and operational incident response measures. The objectives are specified in measurable terms below.

Primary Goal:

Design and implement a simulation framework for Nusuk that evaluates system behavior under cyberattacks and integrates machine-learning-based detection and incident response mechanisms to ensure service continuity.

Measurable Objectives:

1. **Design a Simulation System for Nusuk**
    - *Goal:* Develop a minimal but representative mini-application that models core Nusuk functionalities (e.g., user registration, booking, status display, and API endpoints).
    - *Measurement:* Completed UI prototypes and implemented API endpoints; end-to-end demo of core workflows in the lab environment.
2. **Simulate Cyber-Attacks and Evaluate System Resilience**
    - Goal: Simulate DDoS-related attack scenarios and evaluate system performance under malicious traffic conditions.
    - Measurement: Conduct load and attack tests for each attack type and analyze its impact on availability and performance.
3. **Integrate AI for Attack Detection and Classification**

- *Goal:* Implement and integrate machine learning models (e.g., anomaly detection or supervised classifiers) to detect and classify attack traffic in near real-time.
- *Measurement:* Achieve detection performance with a target **accuracy/recall ≥ 85%** on held-out test data, and report precision, recall, F1-score, and false-positive/false-negative rates.

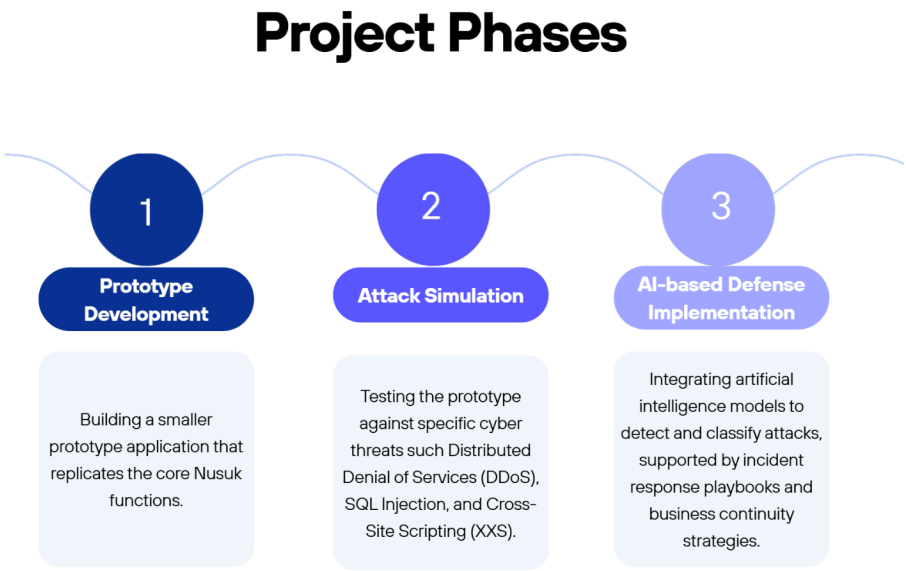


Figure 1 Project Phases

1.4 Project Scope and Limitations

Scope

This project focuses on research and development within a controlled environment and does **not** include live testing against production systems. The deliverables include:

- A prototype mini-application that replicates the primary user flows of Nusuk (registration, booking, status).
  An isolated testbed (virtualized or containerized) configured to represent application servers, databases, and network elements.
- Attack simulation modules capable of reproducing representative load and application-level threats inside the testbed.
- AI-based detection modules and dashboards for monitoring, metrics collection, and analysis.

Limitations and Constraints

- **Ethical and Legal Constraint:** All attack simulations will be performed exclusively within isolated test environments or with explicit written permission for controlled cloud resources. No offensive activities will be conducted against live Nusuk instances or other production systems.
- **Generalisability:** Results derived from the prototype and testbed may not fully generalize to the production Nusuk environment without adaptation; differences in scale, network topology, traffic diversity, and third-party integrations must be considered.
- **Resource Limitations:** The laboratory environment may be constrained by available computing resources, networking bandwidth, and cost budgets, which can limit the maximum feasible attack scale.
- **Data Availability:** Access to authentic production traffic is restricted for privacy and regulatory reasons. Synthetic or publicly available datasets (e.g., CICDDoS) will be used for model training and evaluation when necessary; this may affect detection model performance compared to models trained on actual production traffic.
- **Time Constraints:** The project schedule (academic timeline) may limit the depth of experimentation (e.g., number of attack variants, ensemble defenses) that can be covered within the allocated period.

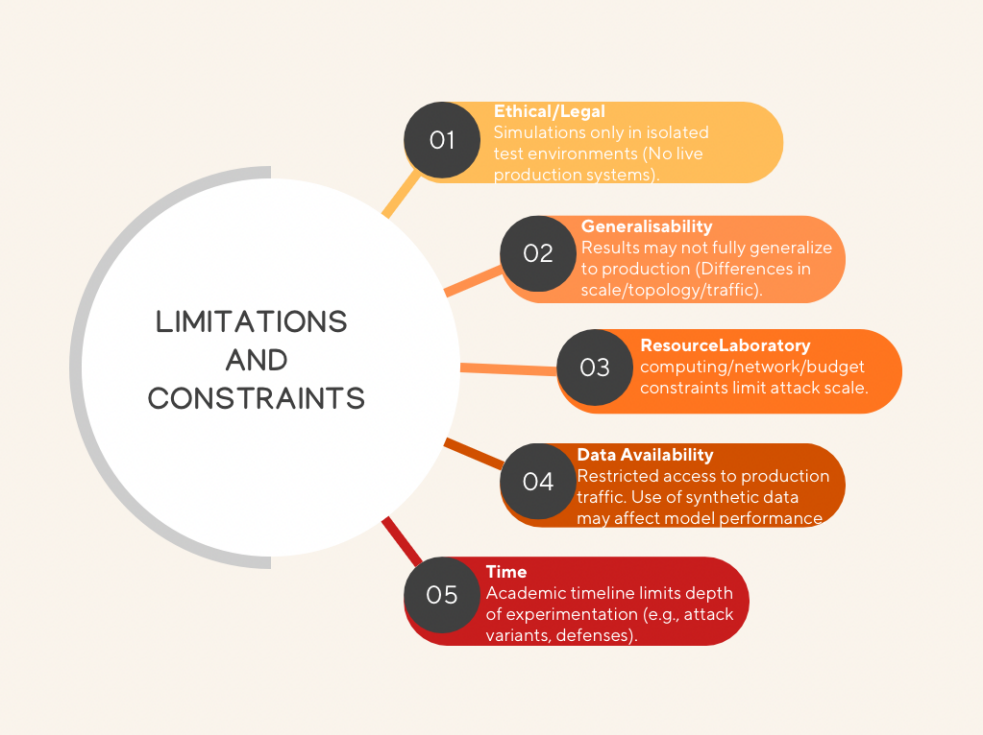


Figure 2 Limitations and Constraints

Chapter 2

Literature Review

2.1 Introduction

This chapter reviews existing research and frameworks relevant to cybersecurity in digital government platforms, particularly pilgrimage applications, and explores the application of artificial intelligence (AI) in intrusion detection and prevention. The aim is to provide a strong theoretical foundation for the project, identify gaps in current research, and assess the feasibility of developing AI-powered defense mechanisms for the Nusuk platform.

The chapter is structured into four main sections:

- **Review of Related Studies :** Reviews existing literature, technologies, and case studies related to cybersecurity in government applications and AI-based intrusion detection.
- **Research Gap and Justification :** Identifies limitations in existing research and technologies, highlighting the need for AI-enhanced security solutions for pilgrimage applications.
- **Feasibility Study:** Examines technical, operational, and financial feasibility for implementing the proposed system.
- **Summary:** Concludes the chapter and introduces the next chapter.

2.2 Review of Related Studies

This section reviews existing literature, frameworks, and case studies relevant to cybersecurity in large-scale digital platforms and AI-based defense mechanisms. Government and public service applications are particularly vulnerable during peak usage periods, as millions of users rely on them simultaneously. Common cyberattacks include Distributed Denial of Service (DDoS), SQL Injection, and data breaches, which can cause applications to crash, freeze, or expose sensitive user data. Recent studies highlight the increasing role of AI in cybersecurity. Techniques such as anomaly detection, intrusion detection systems (IDS), and predictive analytics have been applied

to identify and mitigate attacks in real-time. Existing frameworks provide a foundation, but most research focuses on general enterprise applications rather than government platforms like Nusuk, which have unique challenges related to peak load, user trust, and sensitive personal data protection

2.3 Attacks on Pilgrimage Applications

This section reviews a wide range of cyberattacks targeting pilgrimage applications to provide comprehensive background context. However, not all of these attacks will be addressed within the project scope; only a selected subset (for example, Distributed Denial of Service — DDoS) will be simulated and mitigated as part of the practical implementation .

2.3.1 Distributed Denial of Service (DDoS)

DDoS attacks aim to exhaust some part of a service's capacity — network bandwidth, upstream links, server CPU/memory, or application-layer resources — so legitimate users cannot access the service [1]. While classic volumetric attacks (e.g., UDP floods, amplification/reflection) remain common, threat actors have evolved their playbooks to include application-layer floods (HTTP/HTTPS requests that exhaust server or database resources) and API-targeted tactics that send low-volume but high-impact requests specifically engineered to abuse expensive endpoints or business logic [2], [1] (for example hitting a search endpoint that triggers heavy DB joins). Attackers frequently run short, stealthy probing bursts to map rate-limit thresholds and discover endpoints that bypass simple volumetric controls; once weak points are identified they time a sustained campaign to coincide with peak events (e.g., just before mass registration windows or service announcements). In pilgrimage contexts, the attacker's goal is often to create maximum operational pain [2], [1] during concentrated time windows — taking registration, verification or health-check portals offline for minutes or hours when millions try to access them. Operational telemetry often shows telling signs: sudden repeated patterns of similar requests, spikes in backend latency (DB/CPU) with little change in overall bandwidth, repeated 5xx errors, and many requests from new or geographically dispersed IP ranges [2], [1] that correlate with probing windows These modern DDoS campaigns can combine multi-vector approaches (volumetric + application + API misuse) to force defenders into trade-offs between availability and security [1].

- Mass operational disruption: inability to coordinate pilgrim movement, missed transport windows, delays at checkpoints.
- Financial consequences: lost transactions, refunds, and added mitigation/operational costs.
- Reputational damage undermining trust in official platforms and potentially increasing use of unofficial/unsafe channels.
- Cascading impacts on dependent partners (hotels, transport providers, payment processors) due to systemic outages.

2.3.2 Phishing and Fake-Service Scams — including fake Hajj companies, registration/lottery scams)

Phishing and fake-service scams rely on social engineering and deceptive presentation rather than purely technical exploits. Attackers create emails, SMS/WhatsApp messages, or cloned websites that convincingly imitate official pilgrimage services, travel agencies, or lottery/registration portals. Typical campaigns start with an urgent or aspirational lure e.g., "complete your registration now", "you've won the Hajj lottery - claim your slot", or "your travel documents failed verification" and then direct victims to fraudulent pages that harvest credentials or payment details [3], [4] These scams exploit contextual weaknesses around pilgrimage: language diversity, high emotional/temporal pressure (last-minute confirmations), and reliance on mobile messaging (WhatsApp/SMS). Attackers may combine phishing with hosting fake mobile APKs or malicious attachments (itineraries, manifests), increasing success rates

- Direct financial loss for victims (fraudulent payments, fake bookings).
- Account takeover causing unauthorized changes to itineraries, permits, or health records.
- Large-scale identity theft: aggregated personal records enable secondary scams and targeted attacks.
- Increased operational overhead to investigate incidents, issue refunds, and re-verify legitimate pilgrims.
- Erosion of trust in official communications and platforms [3]

2.3.3 API Abuse and Targeted API Attacks

APIs power mobile and web interactions and connect frontends, partners, and backend services. API abuse exploits weak authentication, missing or coarse rate limits, improper authorization checks, or business-logic flaws. Attackers deliberately craft sequences of API calls that appear legitimate at the packet level but are malicious in aggregate such as systematic account enumeration, high-frequency reads from data-rich endpoints, replayed payment callbacks, or calls crafted to trigger expensive backend processes [5], [1]. Because API traffic often uses valid tokens and mimics

legitimate clients, conventional volumetric detection may not trigger; defenders instead observe anomalous per-endpoint patterns (disproportionate read/write ratios, spikes in 4xx/401 errors, or unusual client IDs) [1].

- Data leakage from API endpoints (lists of pilgrims, booking statuses, partial PII).
- Business logic manipulation leading to fraudulent confirmations, double bookings, or quota exhaustion.
- Corrupted or inconsistent backend records are undermining system integrity.
- Lengthy, complex forensic investigations because malicious sequences resemble normal traffic.
- Potential legal/regulatory consequences if PII is exposed via API exploitation; ReSecurity [2], [1], [5], [6], [7], [8] , [9].

2.3.4 Data Breaches — Database Compromise & Sensitive Data Leaks

Data breaches occur when stored information is exposed through SQL injection, improper access controls, exposed storage buckets, stolen admin credentials, flawed backups, or compromised third parties. Pilgrimage applications typically manage sensitive datasets — passport numbers, national IDs, vaccination records, travel itineraries, and payment data — which are highly valuable on illicit markets. Breaches can be gradual (exfiltration over time) or sudden (bulk export); attackers often throttle reads or mimic backup behaviour to avoid detection. Harvested PII can enable identity theft, targeted phishing, and bypassing of identity checks at checkpoints [8], [7], [3].

- Severe privacy violations and long-term harm to victims (identity theft, travel document misuse).
- Legal, regulatory and financial penalties for data controllers.
- High remediation costs: notifications, monitoring, legal actions, and system re-hardening.
- Operational disruption while records are reconciled and integrity is reestablished.
- Loss of public trust that can reduce adoption of centralized digital pilgrimage services [8], [7]

2.3.5 Man-in-the-Middle (MitM) and Insecure Network Interception

MitM attacks intercept or modify communications between clients and servers. Attackers exploit insecure public Wi-Fi, rogue access points, DNS manipulation, or compromised networking gear to transparently proxy victims' traffic and capture credentials, session cookies, or sensitive files (digital visas, health certificates). Advanced interception techniques (TLS downgrade, forged certificate chains) can inject malicious content (drive-by downloads, JavaScript payloads) without clear user

indication. Pilgrims are particularly vulnerable due to frequent use of transient networks in transit hubs, hotels, or crowded public areas [10], [11], [12].

- Silent credential theft leading to account takeover and unauthorized transactions.
- Manipulation of user-facing data (altered itineraries, falsified confirmations) causes safety and logistical hazards.
- Exposure of sensitive documents in transit increases victims' vulnerability to fraud.
- Difficulty in attribution when interception occurs on third-party network paths.
- Increased verification workload for operators to re-establish legitimate state [11]

2.3.6 Social Engineering — Targeted Messaging and SMShing/WhatsApp Scams

Social engineering leverages psychology rather than technical flaws. Attackers craft messages exploiting urgency, authority, or emotional triggers — impersonating ministry notices, transport updates, or emergency health alerts — to compel recipients to disclose credentials, install attachments, or follow malicious links. In pilgrimage contexts, localized languages, cultural cues, and time pressure (registration deadlines, lottery announcements) increase effectiveness. Sophisticated campaigns use prior data harvesting to tailor messages for higher believability [3], [4] High success rate for compromises due to contextual urgency.

- Rapid lateral spread as compromised users become vectors to contacts and groups.
- Operational delays while teams validate and triage suspected fraudulent messages.
- Challenges in maintaining a single authoritative communication channel during critical events.
- Psychological distress for victims who lose access to essential credentials mid-journey [4], [3]

2.3.7 Exploitation of Application Vulnerabilities (SQLi, XSS, RCE, insecure dependencies)

Application vulnerabilities arise from coding errors, insufficient input validation, insecure configuration, and outdated third-party libraries. Classic web vulnerabilities — SQL Injection (SQLi), Cross-Site Scripting (XSS), Remote Code Execution (RCE) — allow attackers to execute arbitrary queries, run scripts in victims' browsers, or execute code on servers. In microservices and mobile ecosystems, a vulnerability in a

single component (feedback widgets, third-party SDKs, admin consoles) can be leveraged to escalate privileges and move laterally [7], [13] , [14]

- Direct compromise of systems leading to data theft, service disruption, or full server takeover.
- Wide blast radius if a vulnerability exists in a commonly used module or shared component.
- Hidden persistence and later exploitation if attackers gain shell or admin control.
- Significant recovery timelines as systems are audited, cleaned, and patched.
- Loss of confidence among partner organizations that rely on the platform's integrity [7] , [13].

2.3.8 Supply-chain / Third-party Provider Attacks

Supply-chain attacks target trusted third parties — payment processors, notification services, transport partners, cloud analytics providers, or widely used SDKs — integrated into pilgrimage platforms. Attackers compromise a vendor or a signed update/artifact and use that trust relationship to deliver malicious updates, steal tokens/keys, or pivot into the primary platform. Such attacks are stealthy because malicious code or configuration changes may be distributed through legitimate, signed channels; detection is complicated when integrity checks appear to pass [1], [8] , [15].

- Systemic failure or compromise affecting many dependent services simultaneously.
- Widespread data exposure if a trusted vendor holds consolidated records or tokens.
- Severe reputational and operational fallout requiring multi-party coordination to remediate.
- Prolonged incident response due to cross-organizational investigations and contractual implications.
- Regulatory consequences under data-protection and procurement rules [1] , [15]

2.3.9 Malware Distribution via Fake Updates or Malicious Attachments

Malware campaigns distribute trojans, spyware, backdoors, or ransomware by tricking users into installing fake updates or opening malicious attachments that appear legitimate (e.g., itinerary PDFs, manifest ZIPs). Attackers often spear-phish administrative or operational staff with authoritative-looking documents (crew lists, manifests). Once attackers attain a foothold on an admin workstation or CI/CD node,

they can exfiltrate credentials, manipulate records, implant persistent backdoors, or deploy ransomware that encrypts critical systems. Seasonal spikes in messaging (pre-Hajj communications) increase the chance recipients open attachments quickly [4] [3]

- Persistent attacker access enabling long-term data theft or sabotage.
- Exfiltration and manipulation of critical operational records (bookings, manifests).
- Potential encryption of systems and ransom demands, causing major operational disruption during peak pilgrimage windows.
- Elevated recovery costs and potential loss of forensic evidence if backups are corrupted or destroyed.
- Increased operational risk as admin and operator devices become untrustworthy [4] [9]

2.4 Defense Mechanisms Against Application Attacks

2.4.1 Distributed Denial of Service (DDoS) — including API-targeted floods and probing

DDoS attacks target the exhaustion of service capacity, such as network bandwidth, server CPU, or application-layer resources, preventing legitimate users from accessing the service. Modern campaigns use multiple tactics, such as volumetric floods (UDP or amplification attacks), application-layer floods (HTTP/HTTPS requests), and subtle API-targeted attacks that send low-volume requests at high frequency. AI can detect abnormal request patterns by modelling API behaviour, allowing early detection and mitigation of attacks by rerouting traffic to scrubbing centers. Traditional defences include network scrubbing using **CDNs** and **WAFs**, adaptive rate-limiting, and traffic analysis to detect attacks. [1], [16], [17]

2.4.2 Phishing and Fake-Service Scams (including fake companies, registration/lottery scams)

Phishing exploits human trust through seemingly official messages such as emails, SMS, or fake websites mimicking official ones. These attacks often target pilgrims by sending messages claiming they've won a lottery or asking them to confirm registration on fraudulent sites, leading to the theft of sensitive data like account details or payment information. AI can detect fake pages by analyzing visual and textual similarity, as well as examining message metadata to uncover fraud patterns. Traditional defences include public awareness campaigns, multi-channel verification through official sites, and using secure payment gateways [3], [4], [18].

2.4.3 API Abuse and Targeted API Attacks

API abuse occurs when attackers exploit authentication flaws, missing or imprecise rate limits, improper authorization checks, or business logic flaws. These attacks are often subtle, as attackers mimic legitimate client behaviour, sending multiple sequential requests, such as account enumeration or exploiting search endpoints to extract data. AI can create behavioural models for each API endpoint to detect abnormal patterns, such as excessive verification attempts from the same client. Traditional defences include strong authentication using **mutual TLS** or **OAuth with short-lived tokens**, and fine-grained authorization checks to ensure users have proper permissions [5], [6], [15].

2.4.4 Data Breaches — Database Compromise & Sensitive Data Leaks

Data breaches occur when attackers exploit SQL injection, improper access controls, or exposed storage to gain access to sensitive databases and extract information like passport numbers and payment details. AI can detect abnormal data extraction patterns, such as unexplained large reads or unusual activities related to backups, indicating data leakage attempts. Traditional defences include encrypting data at rest and in transit using techniques like **AES** and **TLS**, as well as enforcing multi-factor authentication (MFA) for access to sensitive systems [7], [8], [19].

2.4.5 Man-in-the-Middle (MitM) and Insecure Network Interception

MitM attacks happen when attackers intercept or alter communications between users and servers, allowing them to capture session data, passwords, or other sensitive information. These attacks typically exploit open Wi-Fi or compromised access points. AI can analyze traffic and TLS fingerprints to detect suspicious behaviours like certificate manipulation or unauthorized redirects. Traditional defences include enforcing **TLS** for all communications between users and servers and using **certificate pinning** in mobile apps to prevent certificate manipulation during MitM attacks [11], [12], [20].

2.4.6 Social Engineering, Targeted Messaging, and SMShing/WhatsApp Scams

Social engineering exploits urgency or authority to convince victims to disclose sensitive information. In pilgrimage operations, attackers may impersonate ministry notices or transport updates to steal credentials from pilgrims. AI can analyze message metadata and text to detect fraud, such as messages containing unfamiliar short links or suspicious attachments. On the other hand, traditional defences include public

awareness campaigns and verifying critical communications through multiple channels like SMS and trusted apps. [3], [4], [18], [21]

2.4.7 Exploitation of Application Vulnerabilities (SQLi, XSS, RCE, insecure dependencies)

Exploiting software vulnerabilities such as **SQL Injection** or **XSS** allows attackers to run arbitrary queries, hijack sessions, or execute commands on servers. AI can detect runtime anomalies like SQL injection or XSS attacks by monitoring traffic patterns and analyzing stack traces. Traditional defences include following secure coding practices like **OWASP Top 10**, managing dependencies rigorously, and conducting static and dynamic code analysis to detect vulnerabilities [5], [6], [7], [13], [14].

2.4.8 Supply-chain / Third-party Provider Attacks

Supply-chain attacks happen when attackers exploit vulnerabilities in third-party services, such as payment processors or cloud-based analytics providers, to access the main system. AI can monitor vendor behaviour and detect deviations that may indicate threats. Traditional defences include conducting thorough **vendor security assessments** and using **code signing** to ensure the integrity of third-party data and software [1], [8], [15]

2.4.9 Malware Distribution via Fake Updates or Malicious Attachments

Malware is distributed through fake app updates or malicious attachments, often disguised as legitimate documents. AI can detect suspicious behaviours on endpoints, such as unknown binary execution or unexpected network communications indicative of malware. Traditional defences include **code signing** to ensure update integrity and using **sandboxing** techniques to analyze attachments before execution [8], [9], [12].

Cybersecurity Defences: AI vs. Traditional Methods

Attacks	Description	AI-Based Defence	Traditional Defence
DDoS	Exhausting service capacity via targeted floods	API behaviour modelling, detecting abnormal patterns, rerouting traffic to scrubbing centers.	Network scrubbing using CDNs, rate-limiting, traffic analysis.
Phishing / Fake-Service Scams	Deceptive messages or fake sites to trick users.	Detecting cloned pages, analyzing text and email metadata.	Awareness campaigns, official verification pages, secure payment gateways.
API Abuse / Targeted API Attacks	Exploiting authentication, rate limits, or authorization flaws.	API behaviour modelling, detecting abnormal patterns, challenging suspicious clients.	Strong authentication (TLS/OAuth), fine-grained authorization checks.
Data Breaches	Unauthorized access to databases, leaking sensitive info.	Detecting abnormal data extraction patterns, monitoring backup activities.	Data encryption, multi-factor authentication (MFA).
Man-in-the-Middle (MitM)	Intercepting and altering communications between users and servers.	Traffic analysis to detect certificate manipulation or unauthorized redirects.	Enforcing TLS for all data, certificate pinning in apps.
Social Engineering	Deceiving victims into revealing sensitive info.	Analyzing messages for fraud, recognizing patterns across channels.	Verification through multiple channels, awareness campaigns.
Application Vulnerabilities	Exploiting vulnerabilities like SQLi or XSS to gain unauthorized access.	Runtime monitoring, detecting exploit attempts.	Secure coding practices (OWASP), static/dynamic code analysis.
Supply Chain Attacks	Exploiting third-party services to breach the main system.	Monitoring vendor behaviour, detecting deviations.	Vendor security assessments, code signing for data verification.
Malware via Fake Updates	Distributing malware via fake updates or malicious attachments.	Detecting suspicious behaviours on endpoints, analyzing malware patterns.	Digital signing for updates, using sandboxing techniques to analyze attachments.

Table 1 Cybersecurity Defences: AI vs. Traditional Methods

2.5 Research Gap and Justification

Despite significant advancements in AI-driven cybersecurity, several research gaps remain specifically relevant to pilgrimage and government service applications:

- **Limited focus on AI-powered intrusion detection systems** tailored for government or pilgrimage platforms, where data sensitivity and service availability are equally critical.
- **Existing solutions tend to address either data protection or service availability** in isolation, with few frameworks integrating both dimensions for holistic protection.
- **Insufficient research on peak-load scenarios**, during which cyberattacks such as DDoS are most likely to cause application crashes, slowdowns, or service interruptions.
- **Underexplored integration of predictive analytics** and real-time threat mitigation techniques in mobile-based public applications.

This project aims to bridge these gaps by developing a **prototype system** that simulates the core functionalities of the *Nusuk* application, **tests its performance under various cyberattacks**, and **implements AI-based mechanisms** to enhance detection, mitigation and resilience—particularly during high-traffic periods.



Figure 3 Gaps in Literature

2.6 Feasibility Study

The feasibility of this project is analyzed from three perspectives:

- **Technical Feasibility:** Modern frameworks like **Python, Flask/Django**, and AI libraries such as **TensorFlow** and **Scikit-learn** make it possible to develop and integrate AI-based intrusion detection systems. Simulation tools (e.g., LOIC for DDoS testing) enable testing under controlled conditions.
- **Operational Feasibility:** The project will follow **Agile methodology**, allowing iterative development, continuous feedback, and integration of enhancements based on simulation results.
- **Financial Feasibility:** The project relies on open-source tools and cloud-based platforms, requiring minimal financial investment. Potential challenges include ensuring compliance with data privacy regulations, which will be addressed through encryption, secure authentication, and safe data handling practices.

Overall, the project is practical and achievable within the given constraints.

2.7 Summary

This chapter provided a review of the literature related to cybersecurity challenges in government and pilgrimage applications, highlighted the gaps in existing research, and assessed the feasibility of implementing AI-powered defence mechanisms. The next chapter will outline the Methodology, detailing how the project will simulate cyberattacks on Nusuk, develop AI-based mitigation strategies, and evaluate system performance.

Chapter 3

Methodology

3.1 Research Methodology

This study adopts a **simulation-based experimental methodology** to analyze and evaluate the effectiveness of AI-driven cybersecurity solutions for pilgrimage applications, specifically the *Nusuk* platform. The chosen methodology allows the development of a functional prototype that replicates the core functionalities of the real system, followed by simulated cyberattacks to assess performance and resilience.

The **Agile development methodology** is employed in this project. Agile is selected because it supports iterative development, continuous feedback, and flexibility in responding to changes during system design and testing. Given the evolving nature of cybersecurity threats, Agile allows the team to refine defense mechanisms and integrate new features progressively after each testing phase.

This approach ensures that the system remains adaptable and can efficiently incorporate insights gained during attack simulations and threat analyses.

3.2 Tools and Technologies / Data Collection Methods

The project integrates several tools and technologies across multiple domains to support system development, simulation, AI-based detection, and evaluation. Each category below outlines the technologies selected, their roles, and the rationale for their inclusion.

3.2.1 Frontend Development

- **Figma:** Used to design the graphical user interface (GUI) prototype of the proposed system. Figma was selected because it allows rapid interface prototyping, interactive screen design, and visualization of user interaction flows.
    - The designed interfaces simulate the behavior of the proposed cybersecurity system integrated with the Nusuk platform, including login screens, security dashboards, and attack alert interfaces.
    - Figma also enabled collaborative design, easy modification of UI components, and realistic visualization of how the system would appear in a real-world deployment environment.

3.2.2 Backend and Cloud Services

• **Python:** Used as the primary programming language for implementing machine learning and deep learning models, data preprocessing, performance evaluation, and visualization of

experimental results. Python was selected due to its simplicity, flexibility, and strong support for artificial intelligence and cybersecurity research.

• **Scikit-learn:** Used for implementing traditional machine learning algorithms such as Support Vector Machine (SVM), Random Forest, and Logistic Regression.

• **TensorFlow / Keras:** Used for developing and training the Long Short-Term Memory (LSTM) deep learning model.

• **Pandas and NumPy:** Used for dataset handling, preprocessing, feature manipulation, and numerical computations.

• **Matplotlib:** Used for generating graphs and visualization figures to compare model performance across different experimental scenarios.

• **Pre-prepared Datasets:** Publicly available cybersecurity datasets, including CICDDoS2019 and UNSW-NB15, were used for training and evaluating the implemented models.

- ### 3.2.3 Data Collection Methods:

The data used in this project is sourced primarily from a publicly available and academically recognized intrusion detection dataset. This ensures high-quality, labeled data for training and evaluating the machine-learning model while maintaining ethical and legal compliance.

1.1.1 Public Intrusion Detection Dataset (Primary Data Source)

This project uses the **CICDDoS2019 dataset** [22], a publicly available intrusion detection dataset widely used in cybersecurity research . The dataset contains realistic network traffic generated in a controlled environment and includes both benign flows and multiple families of DoS and DDoS attacks, such as:

- DNS Flood
- LDAP Flood
- MSSQL Flood
- NetBIOS Flood
- NTP Flood
- Portmap Flood
- SYN Flood
- UDP Flood
- UDP-Lag attacks

Each attack type was simulated using real attack tools, increasing the realism and diversity of the dataset.

.2 Dataset Structure

The CICDDoS2019 dataset is divided into multiple training and testing files for each attack category. Examples include:

- *LDAP-training.parquet / LDAP-testing.parquet*
- *DNS-training.parquet / DNS-testing.parquet*
- *SYN-training.parquet / SYN-testing.parquet*

All training files were combined to form the final dataset used to train the machine learning model, while all testing files were merged to evaluate model performance on unseen data.

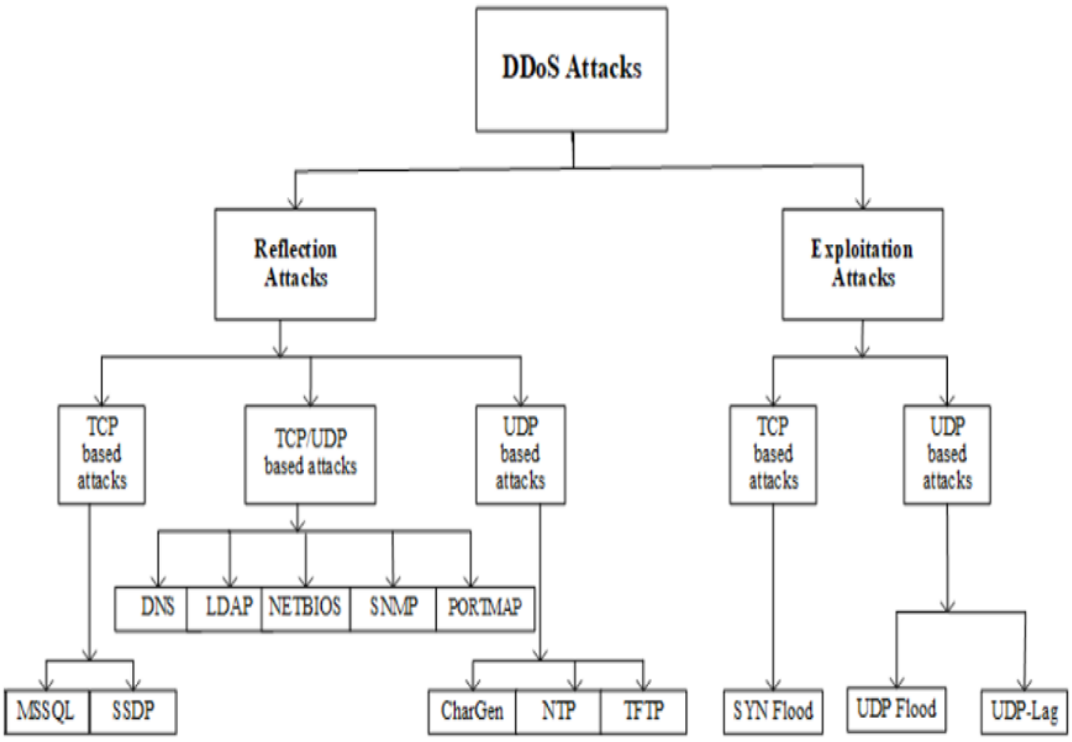


Figure 4 dataset Structure CICDDoS2019

3.٢،٣.3 Dataset Features

Each network flow contains more than 80 numerical features describing various aspects of the communication. These features fall into categories such as:

- **Traffic Volume Features** (e.g., Flow Duration, Total Forward Packets, Flow Bytes/s)
- **Timing Features** (e.g., Inter-Arrival Times)
- **Packet Size Features** (e.g., Packet Length Mean, Variance)
- **TCP/UDP Flag Features** (e.g., SYN, FIN, PSH flag counts)
- **Statistical Behavior Features** (e.g., Active/Idle times, Subflow packets)

Such features enable distinguishing between benign and malicious traffic.

4 Data Transformation

Before model training, labels were converted into numeric values:

- **0 = Benign**
- **1 = Attack**

The dataset was then split into **70% for training** and **30% for validation**, following standard supervised learning practices.

.5 Use of Data in This Project

The processed dataset is used to train and evaluate a **Random Forest classifier**, selected for its:

- Strong performance on tabular network data [23]
- Robustness to noise [24]
- Low susceptibility to overfitting [25]
- Ability to handle large sets of numerical features [24]

The model learns behavioral differences between benign flows and DoS/DDoS attack flows based on the dataset's traffic characteristics.

٣,٢,٣.2 UNSW-NB15

Our project uses the UNSW-NB15 [26] dataset, which is one of the most prominent datasets in intrusion detection research. It was developed at the Cyber Security Research Center at the University of New South Wales (UNSW Canberra) [27]. Network traffic data was created using the IXIA PerfectStorm tool in a controlled environment to combine normal behaviors with modern attack behaviors. Tools such as tcpdump were used to capture approximately 100GB of raw network data, which was then processed and classified using tools like Argus and Bro-IDS into 49 numerical features per network flow.

In this study, the UNSW-NB15 dataset was utilized as a secondary evaluation dataset to assess cross-dataset consistency. The dataset originally contains nine attack categories alongside benign traffic. Since this project focuses specifically on DoS attack detection, the labels were remapped into three categories:**DoS** traffic identified as a Denial of Service attack, **Attack** traffic belonging to any other attack category ,**Non-Attack** benign traffic

This mapping allowed a focused evaluation of whether the implemented models can specifically identify DoS traffic when embedded within a diverse, multi-attack dataset, simulating a more realistic and challenging detection scenario compared to the primary CICDDoS2019 experiments.

2. Dataset Structure

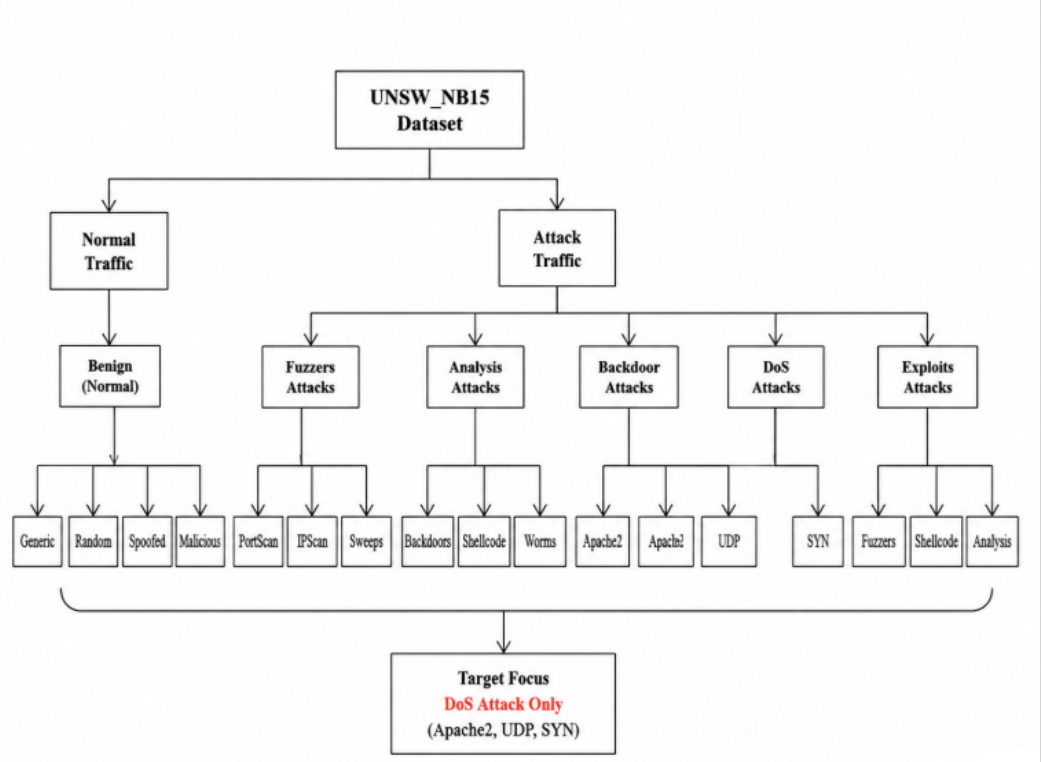


Figure 5 Dataset Structure UNSW NB15

The dataset consists of a collection of files containing network traffic logs, including both benign (normal) data and attack data.

The UNSW-NB15 dataset contains approximately 2,540,044 records spread across several CSV files. Training and testing datasets have been created, such as:

- **UNSW-NB15_training-set.csv**
- **UNSW-NB15_testing-set.csv**

The training dataset contains approximately 175,341 records, while the testing dataset contains about 82,332 records, which include both normal data and attacks**.**

3. Types of Attacks in UNSW-NB15

The UNSW-NB15 dataset contains nine main types of attacks, in addition to the normal class:

- Fuzzers: Attacks aimed at sending unexpected inputs to find vulnerabilities.
- Analysis: Attacks that analyze the system to discover weaknesses.
- Backdoors: Methods to access the system without authorization.

- DoS (Denial of Service): Attacks aimed at disrupting service.
- Exploits: Exploiting known vulnerabilities to take control of the system.
- Generic: General attacks that are not specialized.
- Reconnaissance: Attacks for gathering information about the system.
- Shellcode: Malicious code injected into the system.
- Worms: Malware that self-replicates across the network.

4. **Dataset Features**

Each network traffic record contains 49 numerical features extracted from the network data. These features include:

- Basic features: Protocol, state, and others.
- Time-related features: Flow time, average time between packets.
- Behavioral statistics: Number of packets, data bytes.
- Attack/Behavior features: Indicating attack characteristics in the flow.

5. **Data Transformation and Preparation**

Before training the machine learning model, several preprocessing steps are performed:

- **Label Standardization:**
  - **DoS** (Denial of Service): Any attack labeled as DoS in the attack_cat column was classified as DoS.
  - **Non-Attack**: Any record labeled as Normal in the attack_cat column was classified as Non-Attack, indicating benign network traffic.
  - **Attack**: Any attack other than DoS was classified as Attack, including other types of attacks such as Reconnaissance and Exploits.

  Data Splitting:
  The dataset is divided into:

  - 70% for training
  - 30% for testing/validation

This follows standard practices in supervised classification models.

3.2.4 Cybersecurity Simulation and Testing

• **Python Environment:** The experiments were conducted using Python to implement machine learning and deep learning algorithms, preprocess datasets, and evaluate model performance.

• **VS Code:** Used for executing experiments, training models, monitoring outputs, and visualizing results.

• **Matplotlib:** Used to generate performance comparison graphs and visualization figures for evaluation purposes.

• **Pre-prepared Cybersecurity Datasets:** The experiments relied on publicly available datasets, including CICDDoS2019 and UNSW-NB15, to simulate realistic network traffic and cyberattack scenarios.

• **Machine Learning Libraries:** Scikit-learn and TensorFlow/Keras were used to train and evaluate the implemented detection models under different experimental scenarios.

3.2.5 Artificial Intelligence and Machine Learning

- **Python:** Used for implementing AI-based modules for intrusion detection and anomaly detection.

- **Scikit-learn & TensorFlow:** Frameworks for building and training classification models to distinguish between normal and malicious network traffic.

3.2.6 Version Control and Project Management

- **Git & GitHub:** Manage source code versions, facilitate collaboration, and maintain a structured history of development iterations.
- **Monday.com:** Agile project management tool for task organization, sprint tracking, and milestone monitoring.
- **Google Drive / Notion:** Support documentation sharing, research coordination, and team collaboration.

Category	Tool/Technology	Purpose / Role
Frontend Development	Figma	Used to design the graphical user interface (GUI) prototype of the proposed system.
Backend and Cloud Services	Python	Develop backend simulation engine and API endpoints
	TensorFlow / Keras	Used for developing and training the Long Short-Term Memory (LSTM) deep learning model.
	Matplotlib.	Used to generate performance comparison graphs and visualization figures for evaluation purposes
	Pre-prepared Cybersecurity Datasets	Publicly available cybersecurity datasets, including CICDDoS2019 and UNSW-NB15, were used for training and evaluating the implemented models.
	Machine Learning Libraries	Scikit-learn and TensorFlow/Keras were used to train and evaluate the implemented detection models under different experimental

		scenarios
	Pandas and NumPy	Used for dataset handling, preprocessing, feature manipulation, and numerical computations.
Data Collection Methods	Public Dataset	Primary labeled dataset containing benign, DoS, and DDoS network flows used for machine-learning model training and evaluation.
	Dataset Structure	Enables supervised learning by separating training flows from unseen testing flows.
	Dataset Features	Provide rich behavioral indicators distinguishing normal vs. attack traffic.
	Data Transformation	Prepares the data for supervised machine-learning classification.
	Model Use of Data	Learns traffic patterns and identifies DoS/DDoS flows with high accuracy.
Cybersecurity Simulation and Testing	Python Environment	The experiments were conducted using Python to implement machine learning and deep learning algorithms, preprocess datasets, and evaluate model performance.
	VC Code	Used for executing experiments, training models, monitoring outputs, and visualizing results.
	Matplotlib.	Used to generate performance comparison graphs and visualization figures for evaluation purposes
	Pre-prepared Cybersecurity Datasets	The experiments relied on publicly available datasets, including CICDDoS2019 and UNSW-NB15, to simulate realistic network traffic and cyberattack scenarios.
Artificial Intelligence and Machine Learning	Python	Implement AI modules for intrusion and anomaly detection
	Scikit-learn & TensorFlow	Build and train classification models
Version Control & Project Management	Git & GitHub	Manage source code versions
	Monday.com	Task organization, sprint tracking, milestone monitoring
	Google Drive	Documentation sharing and team coordination

Table 2 Project Tools and Technologies Overview

3.3 Project Timeline

The project is divided into multiple phases to ensure systematic progress from research to implementation:

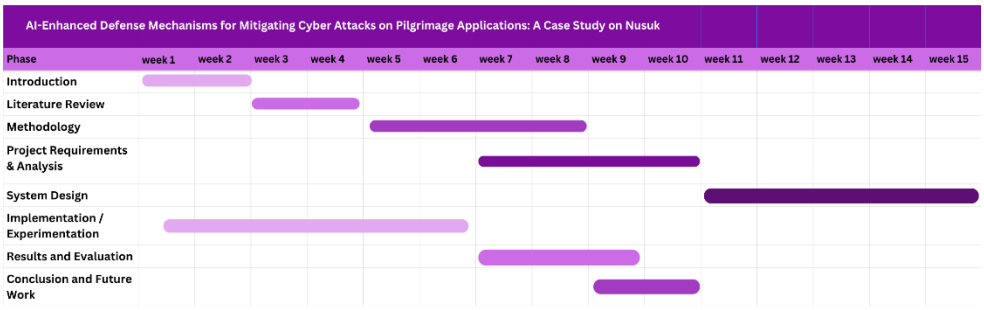


Figure 6 TimeLine

3.4 Risk Management

Effective risk management is a critical component of this project's methodology, ensuring that potential issues are anticipated, assessed, and mitigated before they can affect progress or outcomes. The process follows a structured framework based on the PMBOK Guide [28]which includes risk identification, analysis, response planning, and continuous monitoring.

1. Risk Identification

The primary risks associated with this project fall into four main categories:

- **Technical Risks:** These include potential software bugs, integration issues between modules, or performance bottlenecks during attack simulation.
- **Security Risks:** Given the project's focus on cybersecurity, there is a risk of mishandling sensitive data or inadequate implementation of defense mechanisms.
- **Project Management Risks:** Delays in development phases or miscommunication among team members could affect deliverable timelines.
- **Operational Risks:** Limited hardware resources or internet connectivity issues may impact testing and deployment.

2. Risk Analysis

Each identified risk will be analyzed in terms of its **probability** and **impact**. A qualitative risk assessment matrix will be used to categorize risks as *low*, *medium*, or *high priority*, ensuring that resources are focused on the most critical threats [28].

3. Risk Mitigation Strategies

To minimize risks, the following strategies will be employed:

- **Technical Controls:** Regular system backups, modular coding, and thorough unit testing to prevent data loss and integration failures[27] .
- **Security Controls:** Implementing secure coding practices and access control policies based on the **OWASP Top 10** guidelines [5].
- **Management Controls:** Using Agile sprint reviews and weekly progress reports to maintain schedule alignment and ensure transparency among team members.
- **Operational Controls:** Establishing redundancy plans for servers and maintaining alternative internet access during critical testing phases.

4. Continuous Monitoring

Risk management will not be a one-time process but an **iterative cycle** that aligns with the Agile methodology. Risks will be re-evaluated at the end of each sprint, and new ones will be documented in the **project's risk register**. This continuous assessment approach enables proactive decision-making and enhances project resilience.

In summary, this comprehensive risk management strategy ensures that uncertainties are systematically managed, minimizing potential disruptions and supporting the successful completion of the project.

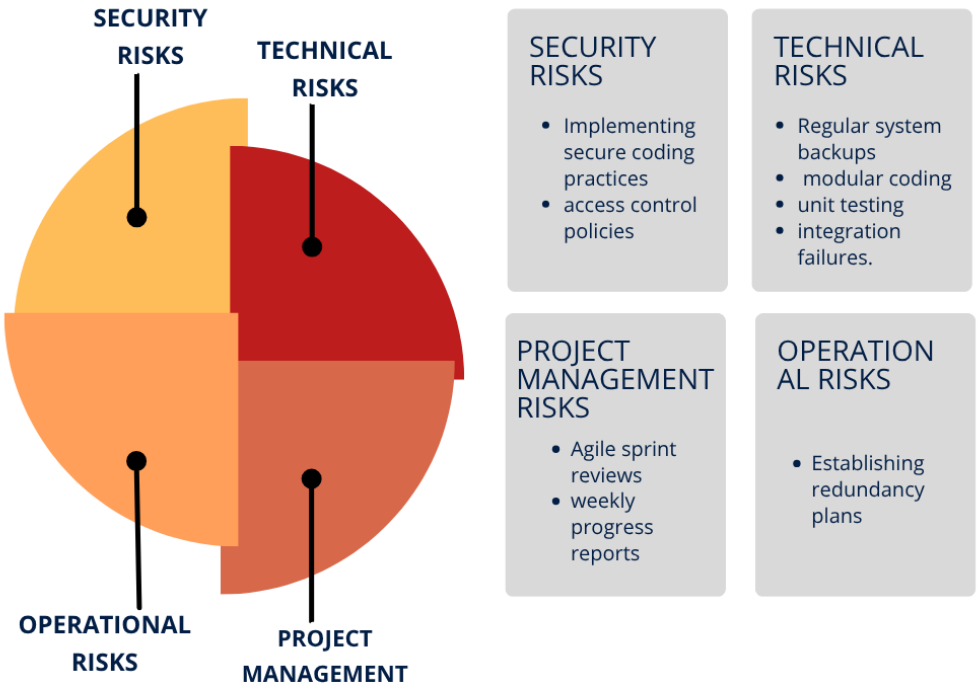


Figure 7 Risk Identification

3.5 Validation and Reliability

Validation in this project focuses on evaluating the accuracy, reliability, and consistency of the machine-learning model used for detecting DoS and DDoS attacks [22] . The validation process follows a structured approach to ensure that the model performs effectively on unseen data and maintains robustness across different traffic conditions.

1. Model Validation Using Unseen Data

A separate **testing dataset**, which contains flows not used during training, was used to evaluate the final performance of the Random Forest classifier.

Using distinct training and testing files ensures that the evaluation remains unbiased and accurately reflects the model's ability to generalize to new traffic patterns.

2. Evaluation Metrics

To assess model performance, the following standard intrusion detection metrics were computed:

- **Accuracy:** Overall correctness of predictions.
- **Precision:** Proportion of predicted attacks that were truly attacks.
- **Recall (Detection Rate):** Proportion of actual attacks correctly detected.

- **F1-score:** Harmonic mean of precision and recall, representing balanced performance.
- **Confusion Matrix:** A visualization of true vs. predicted classes, highlighting false positives and false negatives.

These metrics help measure both the classification strength and the reliability of the detection method.

3. Real-Time Detection and Filtering Validation

After offline evaluation, the trained model was integrated into a lightweight real-time detection workflow. Validation focused on assessing the system's ability to detect and respond to malicious traffic during operation.

a. *Feature Extraction During Operation*

The web server collects real-time traffic statistics, such as:

- Requests per second per IP
- Packet rates
- Abnormally repeated requests
- SYN or UDP flooding behavior

These operational features are converted into the same numerical format used during model training, ensuring consistency.

b. *ML-Based Decision Engine*

For each incoming client or IP address:

- Features are extracted
- They are passed to the trained Random Forest model
- The model outputs one of the two classes:
    - **0 = Normal traffic**
    - **1 = DoS/DDoS attack**

This ensures continuous and automated detection during system operation.

c. Filtering and Mitigation Actions

If an attack is detected, the system can take one or more of the following actions:

- Temporarily block the suspicious IP address
- Drop incoming malicious requests
- Restrict traffic rate
- Create log entries for auditing and security monitoring

These actions validate the system's ability to react promptly and effectively to attack attempts.

3.6 Summary

This chapter presented the research and development methodology used in the project, including the simulation-based experimental approach, Agile framework, tools, and validation strategies.

Through systematic planning and testing, this methodology ensures that the developed prototype not only simulates the Nusuk platform effectively but also provides measurable insights into the performance of AI-based cybersecurity mechanisms under realistic threat conditions.

The next chapter will present the **system design and implementation details** of the proposed solution.

Chapter 4

Project Requirements & Analysis

4.1 Stakeholder Identification

This project involves several stakeholders who play essential roles in the development, analysis, and evaluation of the proposed cybersecurity simulation system for the *Nusuk* platform. The stakeholders and their responsibilities are identified as follows:

Ministry of Hajj and Umrah: Acts as the main beneficiary of the project, responsible for managing and improving the security of digital services provided to pilgrims through the *Nusuk* application.

Developers / Research Team (Students): Responsible for designing the prototype, simulating cyberattacks, analyzing vulnerabilities, and proposing effective defense mechanisms. The team is also in charge of evaluating the system's performance and verifying whether the defined requirements are fully satisfied.

Pilgrims and Visitors: Represent the end users who rely on *Nusuk* for booking and managing Hajj or Umrah services; their data privacy and application access are directly impacted by system security.

Residents in Saudi Arabia: Use *Nusuk* for various local services such as Umrah permits and mosque visit bookings; their inclusion ensures broader usability and security coverage of the system.

Stakeholder	Role
Ministry of Hajj and Umrah	the main beneficiary of the project
Developers / Research Team (Students)	Responsible for designing the prototype, simulating cyberattacks, analyzing vulnerabilities, and proposing effective defense mechanisms. The team is also in charge of evaluating the system's performance and verifying whether the defined requirements are fully satisfied.
Pilgrims and Visitors	Represent the end users

Residents in Saudi Arabia	Use *Nusuk* for various local services such as Umrah permits and mosque visit bookings; their inclusion ensures broader usability and security coverage of the system.

Table 3 Stakeholder Identification

4.2 Methods of Requirements Gathering / Data Needs and Sources

The project adopts a hybrid approach to requirements gathering, combining analytical research methods with the analysis of publicly available cybersecurity datasets. Since the goal of the system is to simulate cyberattack behavior and evaluate an AI-based detection model, the data required for analysis is sourced mainly from established intrusion detection datasets rather than self-generated traffic.

4.2.1 Data Needs and Sources Summary

The system requires several categories of data to support functional analysis, model training, and evaluation:

- **Public Intrusion Detection Dataset:**

The CICDDoS2019 dataset is used as the primary source of labeled benign and malicious network flows. It contains realistic DoS and DDoS attack traffic collected in a controlled environment by cybersecurity researchers.

- **System Interaction Data (Prototype-Level):**

Minimal synthetic data generated internally may be used only to test app functions such as login, booking flow, and alert visualization within the prototype. This data does **not** contribute to model training.

- **Log Data for Testing the System Interface:**

During prototype testing, the system may produce limited logs such as user events, UI actions, and performance indicators. These logs serve usability and interface validation, not AI model training.

- **Detection Output Data:**

After integrating the AI model, the system generates predictions, confidence scores, alerts, and classification results used for evaluation inside the prototype.

All data handling follows ethical standards and excludes any real personal user information.

4.3 Functional Requirements

4.3.1 Functional Requirements for System Administrator / Ministry

Functional Requirement	Description
A1	The Administrator shall be able to log into the system using a secure username and password
A2	The Administrator shall be able to create and manage accounts for technical and cybersecurity teams.
A3	The Administrator shall be able to upload and download system-generated security incident reports.
A4	The Administrator shall be able to execute or evaluate predefined attack-detection scenarios
A5	The Administrator shall be able to view and export system activity logs.

Table 4 Functional Requirements for System Administrator / Ministy

4.3.2 Functional Requirements for End Users (Pilgrims / Residents)

Functional Requirement	Description
U1	The user shall be able to interact with simplified user interfaces that simulate login, service requests, and booking operations.
U2	The user's actions (such as timestamps and events) shall be logged by the system to generate behavioral data for attack scenarios.
U3	The user shall be able to receive reports from the system on service availability and the impact of simulated attacks on the user experience.

Table 5 Functional Requirements for End Users (Pilgrims / Residents)

4.3.3 Functional Requirements for Developers / Research Team

Functional Requirement	Description
D1	The Developers shall have access to a secure sandbox environment for prototype updates.
D2	The Developers shall be able to review generated evaluation results and traffic analysis outputs
D3	The system shall allow developers to add new datasets or evaluation experiments

Table 6 Functional Requirements for Developers / Research Team

4.4 Non-Functional Requirements

4.4.1 Performance & Scalability

Non-Functional Requirements	Description
Response Time	The system shall respond to standard user requests within 2 seconds under normal load.
Throughput	The system should support moderate simulated traffic loads for evaluation purposes.
Scalability	The system shall support horizontal scaling of simulation nodes to accommodate increased load during large-scale attack simulations.
Load Handling	The simulation environment shall gracefully degrade non-critical services under extreme load while preserving core logging and monitoring functionality.

Table 7 Non-Functional Requirements performance and scalability

4.4.2 Availability & Reliability

Non-Functional Requirements	Description
Availability	The experimental environment should remain stable during evaluation runs.
Fault Tolerance	Critical components (logging, monitoring, and controller services) shall have redundancy to tolerate a single node failure without data loss.
Error Handling	The system shall detect, log, and report runtime errors with clear diagnostic messages and timestamps.
Recovery Time Objective (RTO)	In the event of critical failure, the RTO for restoring the simulation controller shall not exceed 30 minutes.

Table 8 Non-Functional Requirements Availability and Reliability

4.4.3 Security & Privacy

Non-Functional Requirements	Description
Authentication & Authorization	All administrative and analyst interfaces shall require authenticated access and role-based authorization
Data Encryption (At-rest)	Sensitive simulation artifacts and logs shall be encrypted at rest using industry-standard algorithms
Data Encryption (In-transit)	All communications between components shall use TLS or equivalent secure channels
Network Isolation	The simulation environment shall be isolated from production networks and the public internet

Access Control Auditing	All access to administrative functions shall be logged with user, timestamp, and action for audit purposes.
Privacy Compliance	Synthetic data shall be used; no real personal data shall be collected or stored. Any required masked datasets must comply with applicable privacy policies.

Table 9 Non-Functional Requirements Security and Privacy

4.4.4 Maintainability & Modifiability

Non-Functional Requirements	Description
Modularity	The system architecture shall be modular to allow independent updates to simulation engine, attack modules, and analytics components.
Testability	The system shall provide automated test hooks or interfaces to run unit and integration tests for attack scenarios and analytics pipelines

Table 10 Non-Functional Requirements Maintainability and Modifiability

4.4.5 Safety & Ethical Considerations

Non-Functional Requirements	Description
Ethical Boundaries	All simulated attacks shall be executed only within the isolated test environment; no real systems shall be targeted.
Risk Assessment	A documented risk assessment shall be prepared for each major experiment, describing potential impacts and mitigation measures.

Table 11 Non-Functional Requirements Safety & Ethical Considerations

4.5 Constraints

The project faces several constraints that may impact the execution and accuracy of the results. These constraints represent factors that may limit the project's ability to achieve its full objectives or affect the quality of the experiments being conducted. Below are the details of these constraints:

1. **Resource Constraints:**
   - The testing environment may be limited in terms of available technical resources, such as processing power (CPUs, memory) and network bandwidth.
   - These limitations could reduce the scale of attacks that can be simulated within the testing environment, affecting the accuracy of simulating attack impacts during peak periods. Additionally, the limited environment might prevent simulating large-scale attacks like DDoS attacks with the same level of realism as they would occur in a production environment.
2. **Ethical and Legal Constraints:**
   - All attack simulations will be conducted in **isolated environments**. No attacks will be executed on live systems or production environments.
   - While these constraints ensure ethical and legal compliance, they may impact the accuracy of results since the simulation does not occur in a live environment with natural traffic or actual user interaction. This may limit the ability to simulate the real-world impact of attacks in production environments.
3. **Time Constraints:**
   - The project is subject to time limitations imposed by the academic schedule and the fixed timeframe allocated for its completion.
   - Due to limited time, it may not be possible to explore all types of attacks or implement every possible defense strategy. This could result in testing only a limited set of attacks or defenses, reducing the comprehensiveness of the evaluation and testing the system under various conditions.
4. **Resource Cost Constraints:**
   - Running large-scale attack simulations may require advanced computational resources and additional costs (such as cloud servers and data storage).
   - If the budget is limited, the project may face difficulty in funding large-scale simulations, which may impact the ability to test **larger attacks** or **multiple defenses** simultaneously.
5. **Tool and Technology Compatibility:**
   - Some tools and technologies used in the testing environment may not be fully compatible with live environments or production systems.
   - This may limit the ability to conduct **comprehensive tests** in the production environment or could lead to discrepancies in the simulation results compared to real-world scenarios.

6. **Data Availability:**
    - The project relies on publicly available datasets for training the AI model (e.g., CICDDoS2019). While these datasets are comprehensive, they may not perfectly represent the exact traffic behavior or attack vectors specific to the Nusuk ecosystem.
    - Public datasets may contain limitations such as class imbalance, incomplete labeling, or differences in feature distributions compared to real-world environments. These factors may affect the model's accuracy when evaluating detection performance under simulation conditions.
    - Since no real user or production network data is used, the realism of the detection results is limited to the characteristics of the dataset. This constraint ensures ethical compliance but reduces the ability to model highly specific scenarios that may occur in operational environments.

4.6 Summary

This chapter presented a comprehensive overview of the system requirements and analysis phase. It began by identifying the key stakeholders involved in the proposed system, including the Ministry of Hajj and Umrah, developers, system administrators, and users interacting with the Nusuk-inspired prototype. Their roles and interests were analyzed to ensure that the proposed system design addresses the needs of each stakeholder effectively.

The chapter also described the methods used for requirements gathering and data analysis. Since the project focuses on AI-based DDoS attack detection and cybersecurity evaluation, publicly available cybersecurity datasets—including CICDDoS2019 and UNSW-NB15—were used for model training and evaluation rather than internally generated network traffic. Prototype-level interactions and generated logs were used only for interface validation and system behavior testing.

In addition, both functional and non-functional requirements were specified in detail. The functional requirements focused on system operations such as attack detection evaluation, traffic analysis, logging, result visualization, and reporting. The non-functional requirements emphasized important system qualities including security, scalability, reliability, usability, maintainability, and ethical compliance to ensure stable and safe experimentation within a controlled environment.

Overall, this chapter established the analytical foundation required for the system design and implementation by clarifying stakeholder needs, data sources, technical limitations, and cybersecurity considerations. This foundation supports the subsequent implementation, experimentation, and evaluation phases presented in the following chapters.

Chapter 5

System Design

5.1 Overview of Design Approach

This chapter presents the system design for the Nusuk simulation system, which is a comprehensive cybersecurity framework that integrates attack simulation, real-time detection, and automated mitigation capabilities. The design approach is based on several key principles that ensure system effectiveness, maintainability, and ethical compliance.

The system employs a **multi-layered defense strategy** with four primary layers: Client Layer, Web Server Layer, ML Detection Layer, and Filtering & Response Layer. This layered architecture follows the defense-in-depth principle, where traffic is processed through multiple stages, each responsible for specific security functions. The system is designed with **modularity** as a fundamental principle, allowing independent development, testing, and deployment of components such as attack simulation, traffic capture, feature extraction, ML detection, and filtering.

A critical design principle is the **complete isolation** of the simulation environment from production systems. The testbed operates in a controlled, isolated network environment that prevents any simulated attacks from affecting real systems or external networks. Additionally, the system integrates **artificial intelligence and machine learning capabilities** for real-time attack detection and classification using the Random Forest algorithm for DoS vs. Normal traffic classification.

The design adopts a hybrid approach, combining modular microservices for independent components with a more integrated architecture for tightly coupled processes. The system implements real-time processing for attack detection and filtering while using batch processing for model training and historical analysis. Security considerations are integrated into every layer, including authentication, authorization, data encryption, audit logging, and input validation.

This chapter will cover the detailed system architecture, data flow analysis, use case diagrams, entity-relationship models, class diagrams, interaction diagrams, activity diagrams, front-end design, database design, and algorithm design. Each section provides a comprehensive view of how the system components work together to achieve the project objectives of simulating cyberattacks, detecting threats in real time, and implementing automated mitigation measures.

5.2 Data Flow Analysis

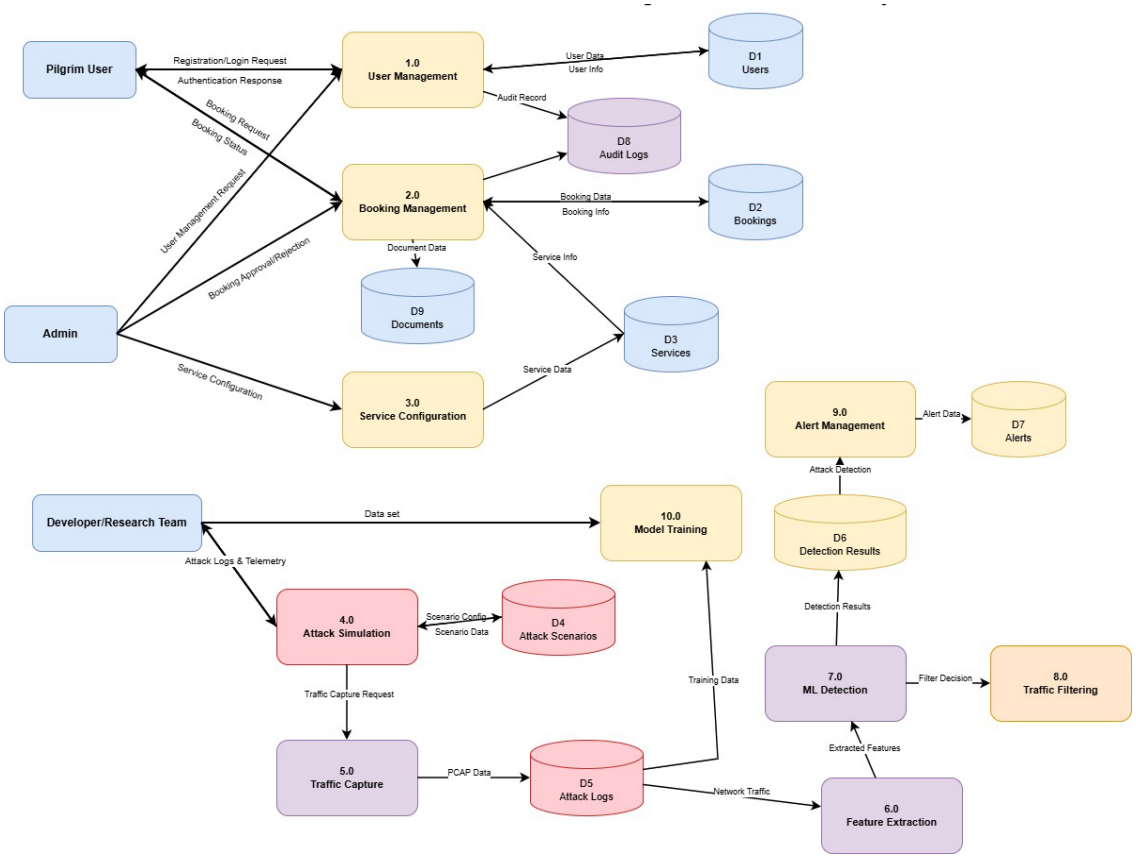


Figure 8 Data Flow

Data Flow Analysis Description

This figure illustrates the overall data flow analysis of the proposed AI-enhanced pilgrimage application security system. It presents how data moves between external entities, core system processes, data stores, and the machine learning components.

At the user level, pilgrims and administrators interact with the system through user management, booking management, and service configuration processes. These processes handle authentication, booking requests, service updates, and document management, while storing and retrieving data from dedicated databases such as users, bookings, services, and audit logs.

The lower part of the diagram focuses on the security and AI pipeline. Network traffic is captured and passed through attack simulation and traffic capture modules, generating traffic and attack logs. These logs are processed through feature extraction and intrusion detection components, where machine learning models analyze traffic patterns to detect malicious activities. Detected results are stored and used for model training and alert management.

Overall, the data flow diagram demonstrates the integration between functional application components and AI-based defense mechanisms, highlighting how operational data and security data are jointly processed to ensure system availability, integrity, and resilience against cyber attacks during high-demand pilgrimage scenarios.

5.3 System Architecture

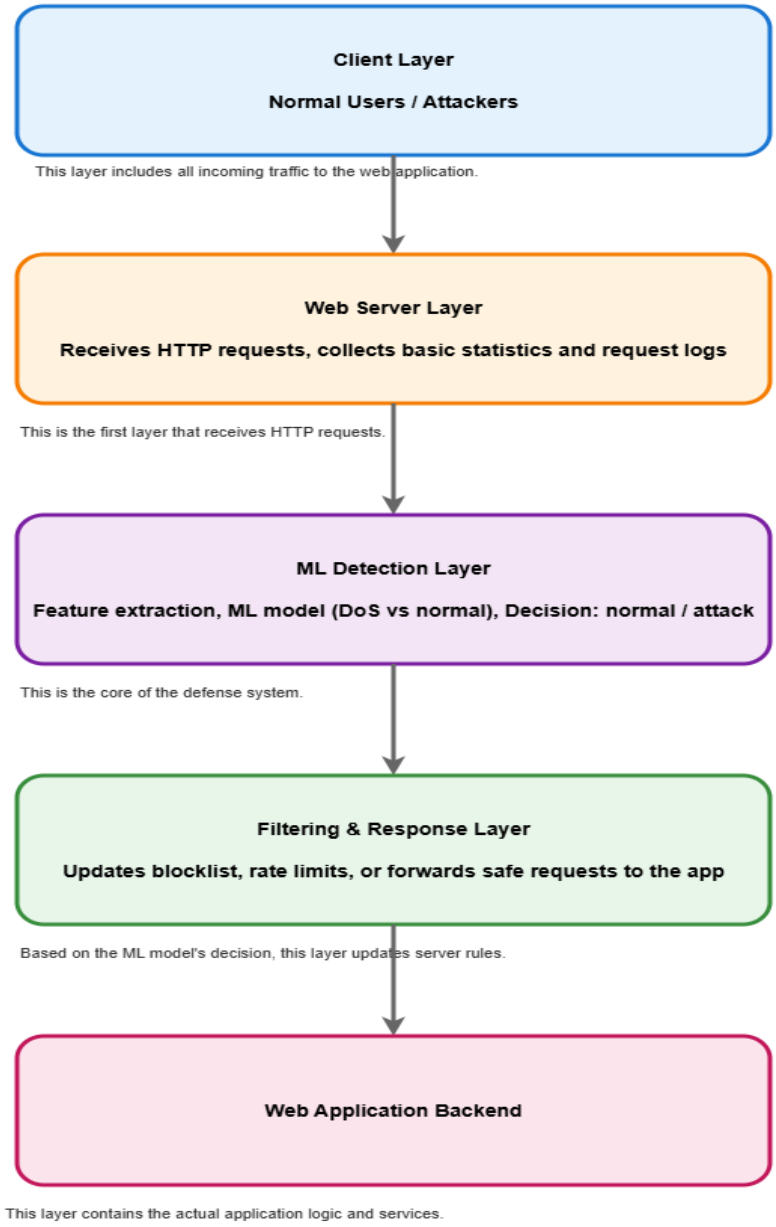


Figure 9 System Architecture

System Architecture Description

The proposed DoS-defense system is designed using four main layers. Each layer performs a specific role to detect and filter malicious traffic before it reaches the backend web application.

1. **Client Layer:**

    - This layer includes all incoming traffic to the web application.
    - Traffic may originate from legitimate users or attackers attempting to overload the server with excessive requests.

2. **Web Server Layer:**

    - This is the first layer that receives HTTP requests.
    - The server records basic statistics such as request rate per IP, repeated URLs, and time between requests.
    - These basic traffic features are forwarded to the ML Detection Layer for analysis.

3. **ML Detection Layer:**

    - This is the core of the defense system.
    - A machine learning model is used to distinguish normal behaviour from DoS attack patterns by analyzing traffic characteristics such as request frequency, packet behaviour, and anomalies.
    - The model outputs a decision: normal or attack.

4. **Filtering & Response Layer:**

    - Based on the ML model's decision, this layer updates server rules.
    - Malicious IPs are temporarily blocked, suspicious traffic is rate-limited, and safe traffic is forwarded to the backend application.

5. **Web Application Backend:**

    - This layer contains the actual application logic and services

5.4 Use Case Analysis

This section identifies the main use cases and actors that interact with the Nusuk simulation system. Use case analysis helps to understand how different users interact with the system and what functions they can perform.

5.4.1 Actors

The system has three main actors that interact with different parts of the system:

Admin

The admin actor is responsible for managing the overall system operations and security functions. Admin has the highest level of access and can Show Alerts, manage users, configure services, approve requests, and monitor security alerts.

Pilgrim / User

The pilgrim or user actor represents the end users who use the system to register, browse services, make bookings, and manage their pilgrimage arrangements. These are the primary users of the Nusuk application functionality.

Developer / Research Team

The developer or research team actor represents the technical staff who work on attack simulation, model training, and system development. They have access to simulation tools, logging functions, and model training capabilities.

5.4.2 Use Cases for Admin

The admin actor can perform the following use cases:

1. **Show Alerts:** Admin can view real-time security alerts and notifications generated by the DoS defense system, including attack detections, system anomalies, and security events. This allows the admin to monitor the system's security status and respond to threats promptly، The admin can also filter alerts by severity, type, date range, status, and source IP, as well as view detailed information related to each alert.
2. **Manage Users:** The admin can create, update, delete, and view user accounts. This includes managing user roles, permissions, and access levels for different types of users.
3. **Configure Services & Slots**: The admin can configure available services, time slots, and service parameters. This includes setting up pilgrimage services, train schedules, and other service offerings.
4. **Approve / Reject Requests:** The admin can review and approve or reject booking requests submitted by users. This ensures that only valid requests are processed.
5. **Manage Bookings:** The admin can view, modify, cancel, and manage all bookings in the system. This allows the admin to handle booking issues and make necessary adjustments.
6. **Audit Logs & Activity:** The admin can view audit logs and system activity records. This helps monitor system usage, track changes, and investigate security incidents.

5.4.3 Use Cases for Pilgrim / User

The pilgrim or user actor can perform the following use cases:

1. **Register:** The user can create a new account by providing personal information and credentials. Registration is required before accessing system services.
2. **Login:** The user can log into the system using their credentials. The login use case extends the register use case, as a user must be registered before logging in.
3. **Browse Services:** The user can browse available pilgrimage services and view service details. This allows users to explore options before making bookings.
4. **Select Time Slot:** The user can select a preferred time slot for their service booking. This use case includes selecting permits and Haramain train services as part of the booking process.
5. **Submit Booking Request:** The user can submit a booking request for selected services and time slots. The request is sent to the admin for approval.
6. **Upload Documents:** The user can upload required documents such as identification papers or permits. These documents may be needed for booking approval.
7. **View Booking Status:** The user can view the current status of their booking requests, including whether the booking is pending, approved, or rejected.
8. **Modify / Cancel Booking:** The user can modify existing booking details or cancel unconfirmed bookings. This provides flexibility for users to manage their plans.

5.4.4 Use Cases for Developer / Research Team

The developer or research team actor can perform the following use cases:

1. **Run Attack Simulation:** The developer can execute attack simulation scenarios to test system defenses. This allows testing of different attack types and intensities.
2. **Configure Attack Parameters:** The developer can configure parameters for attack simulations such as attack intensity, duration, and target endpoints. This provides control over how attacks are simulated.
3. **Add New Attack Scenarios:** The developer can create and add new attack scenarios to the system. This allows expansion of testing capabilities with new attack patterns.
4. **Capture Network Traffic Logs (PCAP):** The developer can capture network traffic in PCAP format for analysis. This provides detailed network data for research and model training.
5. **Export Logs & Telemetry:** The developer can export system logs and telemetry data for analysis. This allows detailed review of system behaviour during attacks.
6. **Monitor Alerts (SIEM):** The developer can monitor security alerts through the SIEM interface. This helps track detected attacks and system responses.
7. **Label Data / Ground Truth:** The developer can label captured data to create ground-truth datasets. This is important for training and validating machine learning models.
8. **Retrain Detection Model:** The developer can retrain the machine learning detection model with new labelled data. This use case includes the label data use case, as labelling is required before retraining.

5.4.5 Use Case Relationships

Some use cases have relationships with other use cases:

- **Login extends Register:** The user must register before logging into the system.
- **Select Time Slot includes Permits and Haramain Train:** When selecting a time slot, the user may also select required permits and Haramain train bookings.
- **Retrain Detection Model includes Label Data:** Before retraining the model, the developer must label the data to create a training dataset.

5.5 System Use Case Diagram

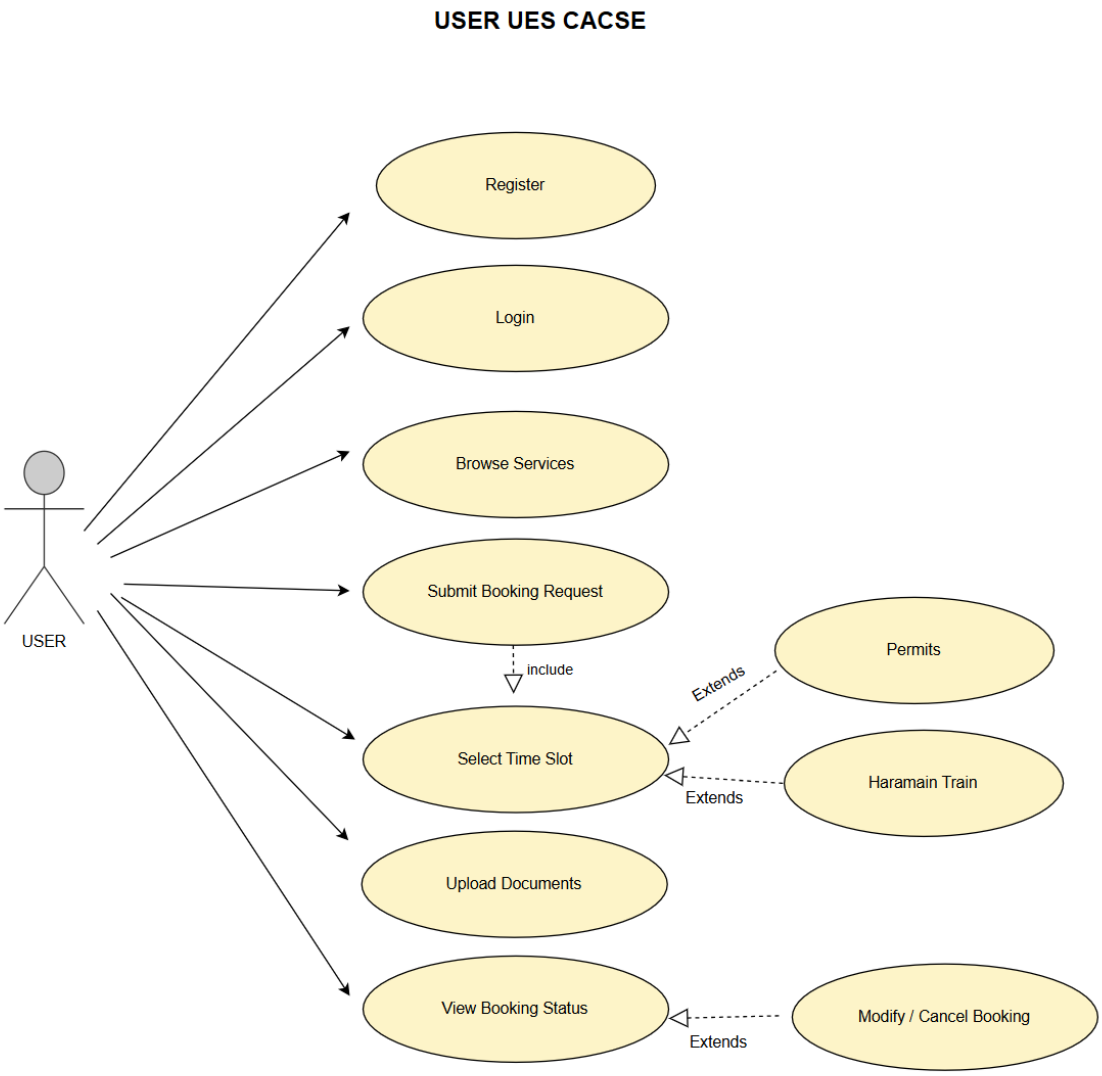


Figure 10 User use case diagram

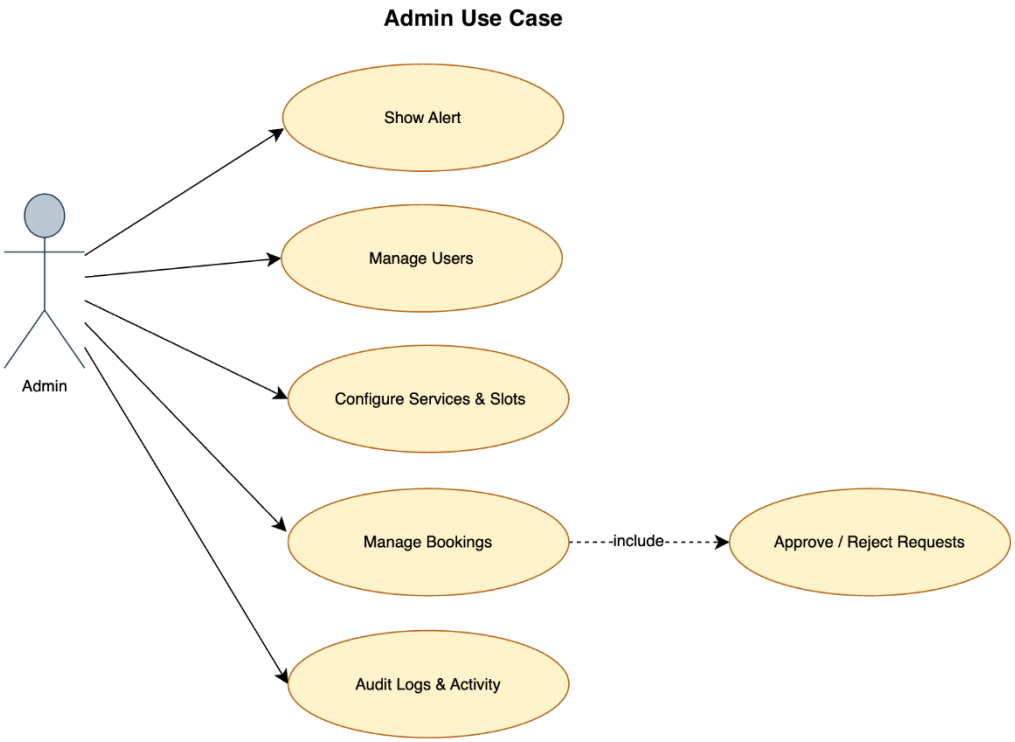


Figure 11 admin use case diagram

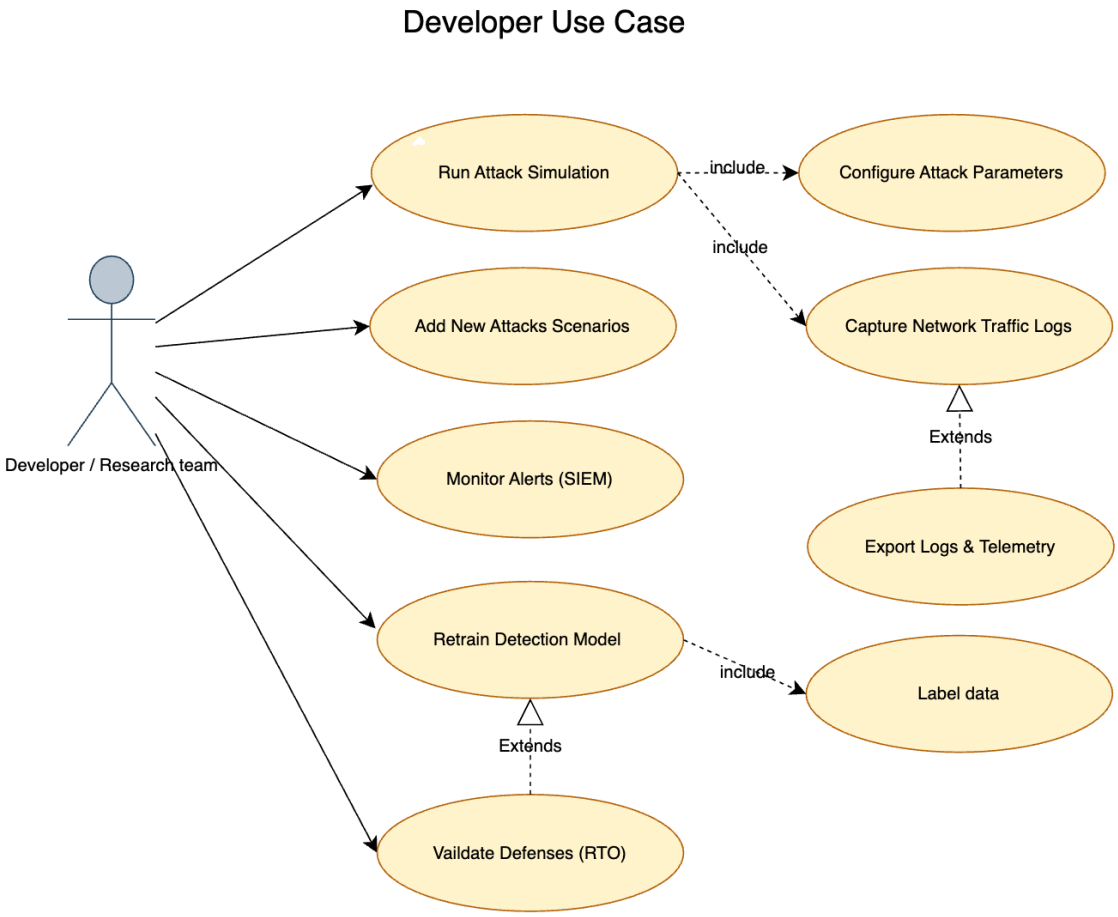


Figure 12 Research Team use case diagram

5.6 System Use Case Description

5.6.1 Pilgrim use case description

Field	Description
Identifier	UC1
Name	User Registration
Actor	Pilgrim / User
Goal	Pilgrim is able to create a new account in the system by providing personal information and credentials to access Nusuk Simulation System services.
Precondition	The application is operational and accessible.
Trigger	The Pilgrim opens the application for the first time or wishes to create a new account.
Postcondition	Pilgrim account is successfully created and stored in the system database.
Main scenario	1. Pilgrim launches application and clicks "Register". 2. System displays registration form. 3. Pilgrim enters personal information, creates username/password, and accepts terms. 4. System validates information and checks for duplicate username/email. 5. System creates account and sends confirmation. 6. System displays success message.
Alternatives	4a. Required field empty: System displays error, returns to form. 5a. Invalid password: System displays password requirements error. 6a. Password mismatch: System displays error. 10a. Username/email exists: System displays error, returns to registration. 9a. Invalid email format: System displays error. 7a. Terms not accepted: System displays error.
Priority	P(1) Critical

Table 12 Pilgrim use case description 1

Field	Description
Identifier	UC2
Name	Secure Access to Pilgrim Services
Actor	Pilgrim / User
Goal	Pilgrim is able to securely log in and access critical route maps, safety information, and service directories.
Precondition	The Pilgrim has successfully registered and the application is operational.
Trigger	The Pilgrim opens the application and wishes to access personalized or secure information.
Postcondition	Pilgrim is successfully authenticated and viewing the main dashboard of services.
Main scenario	1. Pilgrim launches application. 2. System prompts for login credentials (Username/Biometric). 3. Pilgrim provides credentials. 4. System validates credentials and performs security check. 5. System grants access and displays main dashboard.
Alternatives	4a. Invalid credentials: System displays error with remaining attempts, returns to login. 4b. Account locked: System displays lock message, ends use case. 5a. Security alert detected: System displays warning and limits access.
Priority	P(1) Critical

Table 13 Pilgrim use case description2

Field	Description
Identifier	UC3
Name	Browse Available Services
Actor	Pilgrim / User
Goal	Enable the pilgrim to browse and view available pilgrimage services through the Services Home Screen, including access to booking pages (Flights, Hotels, Haramain Train), wallet information, reservations, and permits.
Precondition	Pilgrim is logged into the system.
Trigger	Pilgrim wants to view available services or explore service options.
Postcondition	Pilgrim has viewed available services and their details.
Main scenario	1. Pilgrim successfully logs into the mobile application and lands on the Services Home Screen. 2. System displays the main services page, including: ○ Flights Booking ○ Hotels Booking ○ Haramain Train ○ Nusuk Wallet ○ My Reservations ○ Permits 3. Pilgrim scrolls or browses available services on the page. 4. Pilgrim selects one of the services (e.g., Haramain Train). 5. System displays the detailed service interface containing information such as: ○ Service details ○ Availability ○ Booking options ○ Required documents 6. Pilgrim reviews details and may either: ○ Return back to Services Home Screen, or ○ Continue to booking/confirmation steps through the selected service.
Alternatives	3a. No services available: System displays message, returns to page. 5a. Filter returns no results: System displays message, returns to filter. 6a. Service details unavailable: System displays error, returns to service list.
Priority	P(2) High

Table 14 Pilgrim use case description3

Field	Description
Identifier	UC4
Name	Select Time Slot for Service
Actor	Pilgrim / User
Goal	Pilgrim is able to select a preferred time slot for their service booking, including coordination with Permits and Haramain Train services.
Precondition	Pilgrim is logged in and has browsed available services.
Trigger	Pilgrim wants to book a service and needs to select an available time slot.
Postcondition	Pilgrim has selected a time slot and is ready to proceed with booking submission.
Main scenario	1. Pilgrim selects service from list. 2. System displays available time slots. 3. System checks availability with Permits and Haramain Train systems. 4. System displays integrated time slots with permit/train info. 5. Pilgrim selects preferred date and time slot. 6. System validates and reserves time slot temporarily. 7. System displays confirmation and prompts for booking submission.
Alternatives	2a. No time slots available: System displays message, returns to service list. 3a. Permits/Train system unavailable: System displays warning, continues with available info. 8a. Time slot no longer available: System displays error, returns to selection. 7a. Invalid time slot: System displays error, returns to selection.
Priority	P(1) Critical

Table 15 Pilgrim use case description4

5.6.2 Admin use case description

Field	Description
Identifier	UC1
Name	Show Alerts
Actor	Admin
Goal	Admin is able to view real-time security alerts and notifications generated by the DoS defense system, including attack detections, system anomalies, and security events, to monitor system security status and respond to threats promptly
Precondition	Admin is logged into the system and has appropriate permissions.
Trigger	Admin wants to view security alerts, or system generates new alerts that require admin attention.
Postcondition	Admin has viewed alerts, and alert status may be updated (acknowledged, resolved, or dismissed).
Main scenario	1. Admin accesses alerts dashboard or notifications section. 2. System retrieves and displays list of active alerts from the ML Detection Layer and Filtering & Response Layer. 3. System displays alerts with details including: 4. Admin can sort alerts by timestamp, severity, or type. 5. Admin selects an alert to view detailed information. 6. System displays comprehensive alert details including:, or dismiss alerts. 7. If admin acknowledges or resolves alert, system updates alert status. 8. System logs admin action and updates alert history. 9. System displays confirmation message.
Alternatives	2a. No alerts available: System displays message "No active alerts". 3a. Alert data unavailable: System displays error, retries connection. 4a. Invalid filter criteria: System displays error, returns to default view. 6a. Alert details unavailable: System displays error, returns to alert list. 8a. Alert already resolved: System displays message, no action taken. 9a. System unable to update alert status: System displays error, retries operation. 10a. Logging fails: System continues operation but logs error internally.

Table 16 Admin use case description 1

Field	Description
Identifier	UC2
Name	Manage Users
Actor	Admin
Goal	Admin is able to create, update, delete, and view user accounts, including managing user roles, permissions, and access levels for different types of users.
Precondition	Admin is logged into the system with user management permissions.
Trigger	Admin needs to manage user accounts, roles, or permissions.
Postcondition	User account is created, updated, deleted, or viewed, and changes are saved in the system.
Main scenario	Create User: 1. Admin accesses user management section and clicks "Create User". 2. System displays user creation form. 3. Admin enters user information (name, email, role, permissions). 4. System validates information and checks for duplicate email. 5. System creates user account and sends notification. 6. System displays success message. Update User: 1. Admin selects user from list. 2. Admin clicks "Edit" and modifies user information. 3. System validates changes. 4. System updates user account. 5. System displays success message. Delete User: 1. Admin selects user from list. 2. Admin clicks "Delete" and confirms deletion. 3. System validates user can be deleted. 4. System deletes user account and related data. 5. System displays success message. View Users: 1. Admin accesses user management section. 2. System displays list of all users with filters. 3. Admin can filter by role, status, or search by name/email. 4. Admin selects user to view detailed information.
Alternatives	4a. Duplicate email (Create): System displays error, returns to form. 4a. Invalid information (Update): System displays error, returns to form. 3a. User cannot be deleted: System displays error, deletion cancelled. 5a. No users found: System displays message.

Table 17 Admin use case description 2

Field	Description
Identifier	UC3
Name	Configure Services & Slots
Actor	Admin
Goal	Admin is able to configure available services, time slots, and service parameters, including setting up pilgrimage services, train schedules, and other service offerings.
Precondition	Admin is logged into the system with service configuration permissions.
Trigger	Admin needs to configure or modify services, time slots, or service parameters.
Postcondition	Service configuration is saved and available for users to book.
Main scenario	1. Admin accesses service configuration section. 2. System displays list of existing services or admin creates new service. 3. Admin configures service details (name, type, description, requirements). 4. Admin sets available time slots and dates. 5. Admin configures service parameters (capacity, pricing, duration). 6. Admin configures train schedules if applicable. 7. System validates configuration data. 8. System saves service configuration. 9. System displays success message and updates service availability.
Alternatives	3a. Required fields missing: System displays error, returns to form. 7a. Invalid time slot configuration: System displays error, returns to time slot setup. 7b. Conflicting schedules: System displays warning, admin can override or adjust. 8a. Service name already exists: System displays error, returns to form.
Priority	P(1) Critical

Table 18 Admin use case description 3

Field	Description
Identifier	UC4
Name	Approve / Reject Requests
Actor	Admin
Goal	Admin is able to review and approve or reject booking requests submitted by users, ensuring that only valid requests are processed.
Precondition	Admin is logged into the system and there are pending booking requests.
Trigger	Admin wants to review and process pending booking requests.
Postcondition	Booking request is approved or rejected, and user is notified of the decision.
Main scenario	1. Admin accesses pending requests section. 2. System displays list of pending booking requests. 3. Admin can filter requests by date, service type, or user. 4. Admin selects a request to review. 5. System displays detailed request information (user, service, time slot, documents). 6. Admin reviews request details and documents. 7. Admin clicks "Approve" or "Reject" button. 8. If rejecting, admin enters rejection reason. 9. System validates the action. 10. System updates booking request status. 11. System sends notification to user. 12. System logs the admin action. 13. System displays confirmation message.
Alternatives	4a. No pending requests: System displays message. 6a. Required documents missing: System displays warning, admin can still approve/reject. 8a. Rejection reason not provided: System prompts for reason. 10a. Request already processed: System displays message, no action taken. 7a. Time slot no longer available (Approve): System displays error, admin can reject or suggest alternative.
Priority	P(1) Critical

Table 19 Admin use case description 4

Field	Description
Identifier	UC5
Name	Manage Bookings
Actor	Admin
Goal	Admin is able to view, modify, cancel, and manage all bookings in the system, allowing admin to handle booking issues and make necessary adjustments.
Precondition	Admin is logged into the system with booking management permissions.
Trigger	Admin needs to view, modify, or manage bookings in the system.
Postcondition	Booking is viewed, modified, cancelled, or managed, and changes are reflected in the system.
Main scenario	View Bookings: 1. Admin accesses booking management section. 2. System displays list of all bookings with filters. 3. Admin can filter by status, date, service, or user. 4. Admin selects booking to view detailed information. Modify Booking: 1. Admin selects booking and clicks "Modify". 2. System displays booking details in editable form. 3. Admin modifies booking information (time slot, service, user). 4. System validates changes and checks availability. 5. System updates booking record. 6. System sends notification to user. 7. System displays success message. Cancel Booking: 1. Admin selects booking and clicks "Cancel". 2. System displays cancellation confirmation. 3. Admin enters cancellation reason and confirms. 4. System updates booking status to "Cancelled". 5. System releases resources and sends notification to user. 6. System displays success message.
Alternatives	3a. No bookings found: System displays message. 4a. Booking cannot be modified: System displays error. 5a. New time slot unavailable (Modify): System displays error, returns to modification. 3a. Booking cannot be cancelled: System displays error. 4a. User cancels action: System returns to booking list.
Priority	P(2) High

Table 20 Admin use case description 5

Field	Description
Identifier	UC6
Name	Audit Logs & Activity
Actor	Admin
Goal	Admin is able to review system activity and audit logs to monitor user actions, configuration changes, attack simulation events, and booking operations for security, compliance, and troubleshooting purposes.
Precondition	Admin is logged into the system with appropriate audit and monitoring permissions.
Trigger	Admin wants to review or investigate system activities, verify recent changes, or analyze incidents.
Postcondition	Relevant audit logs and activities are displayed, reviewed, and optionally exported or saved for further investigation.
Main scenario	1. Admin accesses the “Audit Logs & Activity” section from the admin dashboard. 2. System displays a list of recent audit log entries sorted by timestamp. 3. Admin selects filter criteria (e.g., date range, user, event type, severity, scenario, or module). 4. System applies the selected filters and refreshes the audit log list. 5. Admin reviews the filtered list and selects a specific log entry to view details. 6. System displays detailed information for the selected entry (timestamp, user, action performed, affected entity, IP address/location if available, status, and related scenario or module). 7. Admin navigates between log entries as needed to understand the sequence of actions or events. 8. Admin chooses to export the filtered audit logs (e.g., to CSV/PDF) for reporting or investigation. 9. System generates the export file and confirms that the export has completed successfully. 10. Admin returns to the admin dashboard or continues reviewing additional logs.
Alternatives	3a. Invalid filter values: System displays a validation error message (e.g., invalid date range), clears invalid fields, and prompts admin to correct the filters. 4a. No logs match filter criteria: System displays message: “No audit records found for the selected criteria” and keeps the filters applied. 6a. Log entry archived or unavailable: System displays message indicating that detailed information is no longer available for this entry and returns admin to the log list. 8a. Export operation fails: System displays error message (e.g., “Unable to export logs due to system error”), logs the failure, and suggests the admin to retry later.
Priority	P(2) High

Table 21 Admin use case description 6

5.6.3 Research Team use case description

Field	Description
Identifier	UC1
Name	Run Attack Simulation
Actor	Developer / Research Team
Goal	Developer is able to execute attack simulation scenarios to test system defenses and test different attack types and intensities.
Precondition	Developer is logged into the system and attack scenarios are available.
Trigger	Developer wants to execute an attack simulation to test system defenses.
Postcondition	Attack simulation is executed and results are available for analysis.
Main scenario	1. Developer accesses attack simulation dashboard. 2. System displays available attack scenarios. 3. Developer selects an attack scenario. 4. Developer clicks "Run Simulation" button. 5. System validates scenario and checks system resources. 6. System executes attack simulation in isolated testbed environment. 7. System displays simulation progress and status. 8. System completes simulation and displays results. 9. Developer can view detailed results and metrics.
Alternatives	3a. No scenario selected: System displays error, returns to selection. 5a. System resources unavailable: System displays error, simulation fails. 5b. Invalid scenario: System displays error, returns to selection. 6a. Simulation fails: System displays error, returns to dashboard.

Table 22 Research Team use case description 1

Field	Description
Identifier	UC2
Name	Configure Attack Parameters
Actor	Developer / Research Team
Goal	Developer is able to configure parameters for attack simulations such as attack intensity, duration, and target endpoints to control how attacks are simulated.
Precondition	Developer is logged into the system with parameter configuration permissions.
Trigger	Developer wants to configure or modify attack simulation parameters.
Postcondition	Attack parameters are configured and saved for use in simulations.
Main scenario	1. Developer accesses attack parameter configuration section. 2. System displays parameter configuration interface. 3. Developer selects attack type from available options. 4. Developer sets attack intensity level (1-10). 5. Developer sets attack duration. 6. Developer selects target endpoints. 7. Developer clicks "Save Configuration" button. 8. System validates configuration parameters.
Alternatives	4a. Invalid intensity value: System displays error, returns to intensity setting. 5a. Invalid duration value: System displays error, returns to duration setting. 6a. Invalid target endpoints: System displays error, returns to endpoint selection. 8a. Configuration validation fails: System displays error, returns to form.

Table 23 Research Team use case description 2

Field	Description
Identifier	UC3
Name	Add New Attack Scenarios
Actor	Developer / Research Team
Goal	Developer is able to create and add new attack scenarios to the system to expand testing capabilities with new attack patterns.
Precondition	Developer is logged into the system with scenario creation permissions.
Trigger	Developer wants to create and add a new attack scenario to the system.
Postcondition	New attack scenario is created and added to available scenarios list.
Main scenario	1. Developer accesses attack scenario management section. 2. Developer clicks "Add New Scenario" button. 3. System displays new scenario creation form. 4. Developer enters scenario name and description. 5. Developer selects attack type. 6. Developer uploads or configures attack script/configuration. 7. Developer clicks "Save Scenario" button. 8. System validates scenario data.
Alternatives	4a. Scenario name blank: System displays error, returns to form. 4b. Scenario name already exists: System displays error, returns to name field.

Table 24 Research Team use case description 3

Field	Description
Identifier	UC4
Name	Capture Network Traffic Logs (PCAP)
Actor	Developer / Research Team
Goal	Developer is able to capture network traffic in PCAP format for analysis, providing detailed network data for research and model training.
Precondition	Developer is logged into the system with network capture permissions.
Trigger	Developer wants to capture network traffic for analysis or research.
Postcondition	Network traffic is captured in PCAP format and saved for analysis.
Main scenario	1. Developer accesses network traffic capture section. 2. System displays capture interface with available interfaces. 3. Developer selects capture interface or target. 4. Developer sets capture duration or packet count limit. 5. Developer clicks "Start Capture" button. 6. System starts capturing network traffic. 7. System displays capture progress and statistics. 8. Developer clicks "Stop Capture" or capture completes automatically. 9. System saves captured traffic in PCAP format. 10. System displays confirmation with file location.
Alternatives	3a. No capture interface available: System displays error, returns to interface selection. 4a. Invalid capture parameters: System displays error, returns to parameter setting. 6a. Insufficient disk space: System displays error, capture fails. 8a. Capture fails: System displays error, returns to capture interface.

Table 25 Research Team use case description 4

Field	Description
Identifier	UC5
Name	Export Logs & Telemetry
Actor	Developer / Research Team
Goal	Developer is able to export system logs and telemetry data for analysis, allowing detailed review of system behavior during attacks.
Precondition	Developer is logged into the system and logs/telemetry data are available.
Trigger	Developer wants to export logs and telemetry data for analysis.
Postcondition	Logs and telemetry data are exported in selected format and available for download.
Main scenario	1. Developer accesses log export section. 2. System displays export interface with filters. 3. Developer selects date range or time period. 4. Developer selects log types to export (system logs, attack logs, telemetry). 5. Developer selects export format (CSV, JSON, TXT). 6. Developer clicks "Export" button. 7. System validates export parameters. 8. System generates export file. 9. System provides download link or saves file. 10. System displays confirmation message.
Alternatives	3a. Invalid date range: System displays error, returns to date selection. 7a. No data available for period: System displays message, returns to date selection. 8a. Export fails: System displays error, returns to export interface. 9a. File generation fails: System displays error.
Priority	P(2) High

Table 26 Research Team use case description 5

Field	Description
Identifier	UC6
Name	Monitor Alerts (SIEM)
Actor	Developer / Research Team
Goal	Developer is able to monitor security alerts through SIEM interface to track detected attacks and system responses.
Precondition	Developer is logged into the system and SIEM system is operational.
Trigger	Developer wants to monitor security alerts and system events.
Postcondition	Developer has viewed and monitored security alerts through SIEM interface.
Main scenario	1. Developer accesses SIEM monitoring interface. 2. System displays real-time security alerts and events. 3. Developer can filter alerts by severity, type, or time period. 4. Developer selects an alert to view details. 5. System displays detailed alert information (timestamp, source, type, severity). 6. Developer reviews alert details and system response actions. 7. Developer can mark alert as reviewed or take additional actions. 8. System updates alert status.
Alternatives	2a. SIEM system unavailable: System displays error, returns to dashboard. 3a. No alerts found: System displays message. 5a. Alert details unavailable: System displays error, returns to alert list.
Priority	P(1) Critical

Table 27 Research Team use case description 6

Field	Description
Identifier	UC7
Name	Label Data / Ground Truth
Actor	Developer / Research Team
Goal	Developer is able to label captured data to create ground truth datasets, which is important for training and validating machine learning models.
Precondition	Developer is logged into the system and data is available for labeling.
Trigger	Developer wants to label data to create ground truth datasets.
Postcondition	Data is labeled and ground truth dataset is created.
Main scenario	1. Developer accesses data labeling section. 2. System displays available datasets for labeling. 3. Developer selects dataset or data file to label. 4. System displays data samples for labeling. 5. Developer reviews each data sample. 6. Developer assigns label (e.g., Normal, Attack, Attack Type). 7. Developer clicks "Save Label" button. 8. System saves label for the data sample. 9. Developer repeats steps 5-8 for remaining samples. 10. Developer clicks "Complete Labeling" button. 11. System validates all labels are assigned. 12. System creates ground truth dataset. 13. System displays confirmation message.
Alternatives	3a. No data available: System displays message, returns to dashboard. 11a. Not all samples labeled: System displays error, returns to labeling. 6a. Invalid label: System displays error, returns to label assignment.

Table 28 Research Team use case description 7

Field	Description
Identifier	UC8
Name	Retrain Detection Model
Actor	Developer / Research Team
Goal	Developer is able to retrain the machine learning detection model with new labeled data, including the label data use case as labeling is required before retraining.
Precondition	Developer is logged into the system and labeled dataset (ground truth) is available.
Trigger	Developer wants to retrain the detection model with new labeled data.
Postcondition	Detection model is retrained and new model version is available.
Main scenario	1. Developer accesses model training section. 2. System displays available labeled datasets. 3. Developer selects labeled dataset (ground truth) for training. 4. Developer configures training parameters (learning rate, epochs, etc.). 5. Developer clicks "Start Training" button. 6. System validates dataset and parameters. 7. System starts model training process. 8. System displays training progress and metrics. 9. System completes training and evaluates model performance. 10. System displays training results and accuracy metrics. 11. Developer reviews results. 12. Developer clicks "Deploy Model" or "Save Model" button. 13. System saves or deploys new model version.
Alternatives	3a. No labeled dataset available: System displays error, redirects to labeling. 6a. Dataset insufficient or corrupted: System displays error, returns to dataset selection. 7a. Training parameters invalid: System displays error, returns to parameter configuration. 9a. Model training fails: System displays error, returns to training setup.

Table 29 Research Team use case description 8

Field	Description
Identifier	UC9
Name	Validate Defenses (RTO)
Actor	Developer / Research Team
Goal	Developer is able to validate defense mechanisms and measure recovery time objectives (RTO) to evaluate effectiveness of security measures.
Precondition	Developer is logged into the system and defense mechanisms are available for validation.
Trigger	Developer wants to validate defense mechanisms and measure RTO.
Postcondition	Defense mechanisms are validated and RTO metrics are calculated and available.
Main scenario	1. Developer accesses defense validation section. 2. System displays available defense mechanisms. 3. Developer selects defense mechanism to validate. 4. Developer initiates test attack or selects existing attack simulation results. 5. Developer clicks "Start Validation" button. 6. System executes validation test. 7. System measures defense response time and effectiveness. 8. System calculates Recovery Time Objective (RTO). 9. System records validation metrics and results. 10. System displays validation report with RTO metrics and defense effectiveness scores. 11. Developer reviews validation results. 12. Developer can export or save validation report.
Alternatives	3a. No defense mechanism selected: System displays error, returns to selection. 4a. No test attack available: System displays error, redirects to attack simulation. 6a. Validation test fails: System displays error, returns to validation setup. 8a. RTO calculation fails: System displays error, returns to validation.

Table 30 Research Team use case description 9

5.7 Entity-Relationship (ER) Analysis

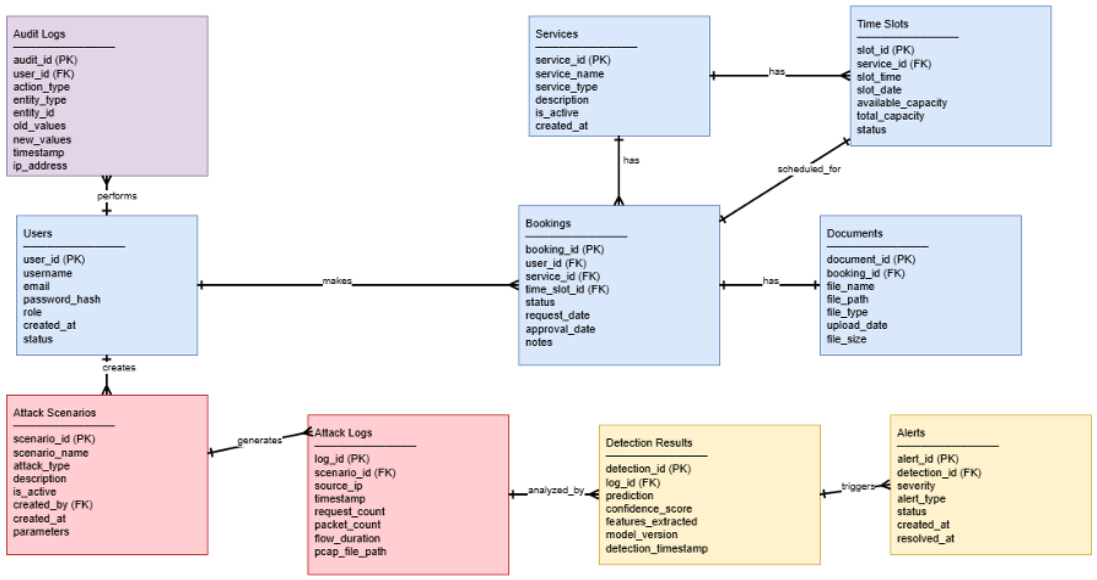


Figure 13 Entity-Relationship (ER)

Entity-Relationship (ER) Description

This figure presents the Entity-Relationship (ER) diagram of the proposed AI-enhanced pilgrimage application and its security monitoring framework. The diagram illustrates the main entities, their attributes, and the relationships that define how operational data and security-related data are stored and interconnected.

Core system entities include **Users**, **Services**, **Bookings**, **Time Slots**, and **Documents**. Users can create bookings for specific services, which are scheduled within defined time slots and may have associated documents uploaded as part of the booking process. Each service can have multiple time slots, enabling capacity management and scheduling control. User activities within the system are recorded in **Audit Logs**, ensuring traceability and accountability for administrative and operational actions.

On the security side, the ER diagram models the attack detection and monitoring components. **Attack Scenarios** define simulated or observed cyber attack configurations and generate corresponding **Attack Logs**, which capture detailed traffic and request information. These logs are analyzed by the detection system to produce **Detection Results**, including prediction outcomes, confidence scores, and extracted features. Based on detection results, **Alerts** are generated to notify administrators of potential security incidents, categorized by severity and status.

Overall, this ER diagram demonstrates a unified database design that supports both the functional requirements of the pilgrimage application and the AI-based intrusion detection process, enabling efficient data management, auditability, and proactive cyber threat response.

5.8 System Class Diagram

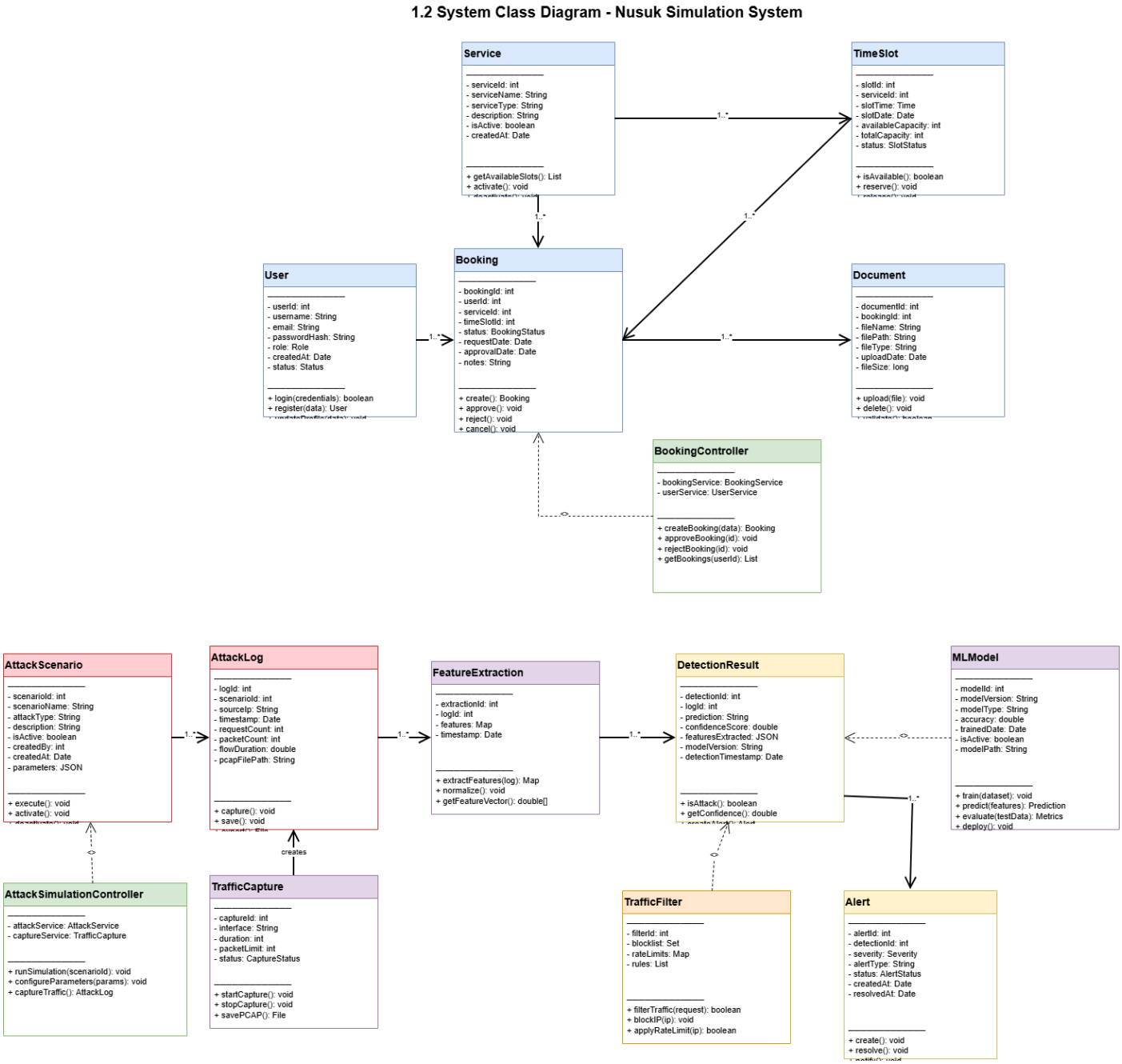


Figure 14 System Class Diagram

System Class Description

This figure illustrates the system class diagram of the proposed Nusuk simulation system, which integrates core booking functionalities with AI-based security and intrusion detection

mechanisms. The diagram represents the structural design of the system by defining the main classes, their attributes, operations, and relationships.

At the application domain level, the system includes core entities such as **User**, **Service**, **Booking**, **TimeSlot**, and **Document**. Users interact with the system by creating and managing bookings for specific services, which are scheduled within predefined time slots. Each booking maintains its lifecycle through different states and may have associated documents uploaded to support the booking process. These classes collectively model the essential business logic of the pilgrimage service workflow.

To manage system interactions and enforce application logic, controller and service-level classes are introduced. For example, the **BookingController** coordinates user requests related to booking operations, acting as an interface between the application layer and the underlying domain logic. This separation supports better organization of responsibilities and improves system maintainability.

The lower part of the diagram focuses on the security and monitoring components of the system. **AttackScenario** and **AttackLog** classes represent simulated or observed cyber attack activities and their corresponding traffic records. Network traffic is captured and processed through components such as **TrafficCapture** and **FeatureExtraction**, which prepare data for analysis. The **MLModel** class encapsulates the machine learning detection logic, generating **DetectionResult** objects that include prediction outcomes and confidence measures. Based on these results, the **Alert** class represents security notifications generated to inform administrators of potential threats.

Overall, this class diagram demonstrates a layered and modular system structure that separates business functionality from security and AI-driven analysis. The design supports extensibility by allowing security mechanisms to evolve independently from core booking operations, while still enabling seamless integration between functional services and intelligent defense components.

5.9 System Interaction Diagram

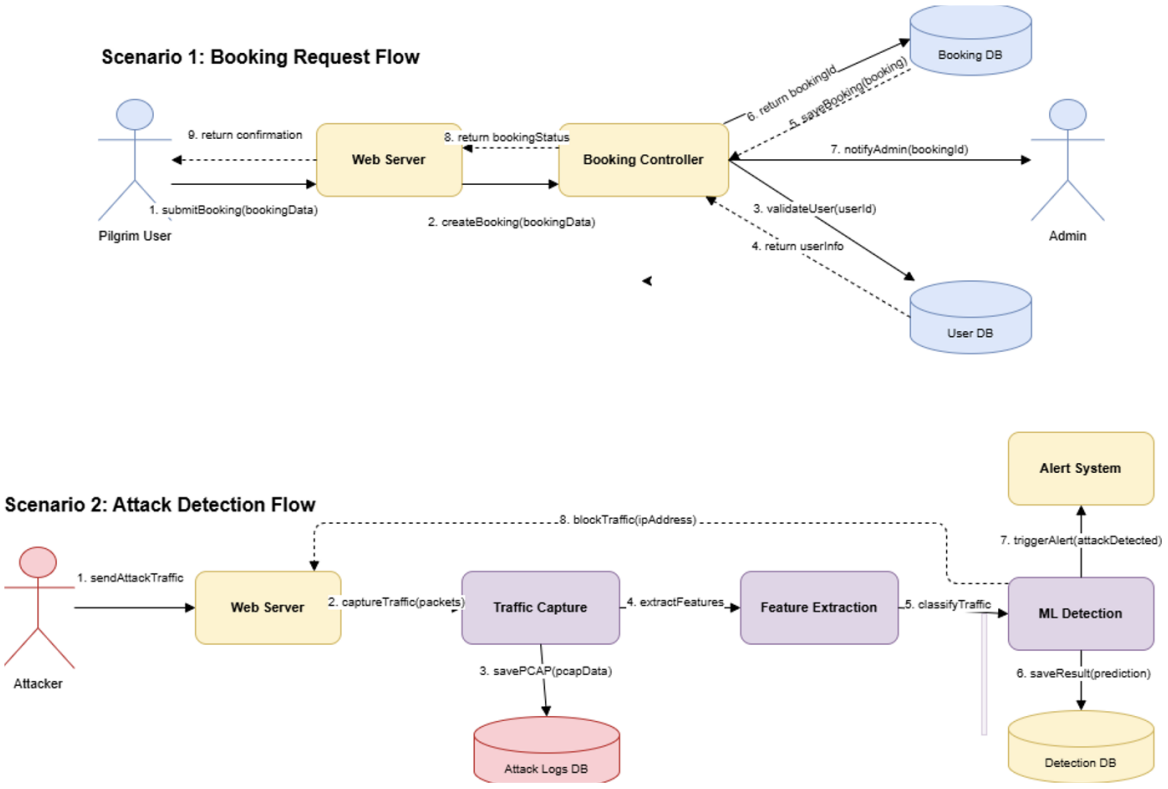


Figure 15 System Interaction Diagram

This diagram illustrates the system's operational flows, detailing the step-by-step data exchange for both user booking requests and the automated attack detection process.

2 Scenario 1: Booking Request Flow Description

The process begins when a Pilgrim User initiates a booking request by submitting data through the Web Server, which then routes the information to the Booking Controller. To ensure security and data integrity, the controller interacts with the User DB to validate the user's identity and retrieve necessary profile information. Once the user is verified, the controller sends the booking details to the Booking DB for permanent storage, receiving a unique bookingId in return. The cycle concludes with two parallel actions: the Admin is notified of the new entry, and a final confirmation status is sent back to the user through the Web Server.

3 Scenario 2: Attack Detection Flow Description

This scenario illustrates the system's automated defense mechanism against malicious activities, starting when an Attacker sends malicious traffic to the Web Server. The system immediately activates the Traffic Capture component to intercept the packets, which are then stored as PCAP data within the Attack Logs DB for forensic analysis. Subsequently, the Feature Extraction module processes the raw traffic and passes the relevant characteristics to the ML Detection engine for classification. Upon identifying a threat, the system saves the prediction results in the Detection DB, triggers the Alert System, and issues a direct command to the Web Server to block the attacker's IP address, effectively neutralizing the threat.

5.10 System Activity Diagram

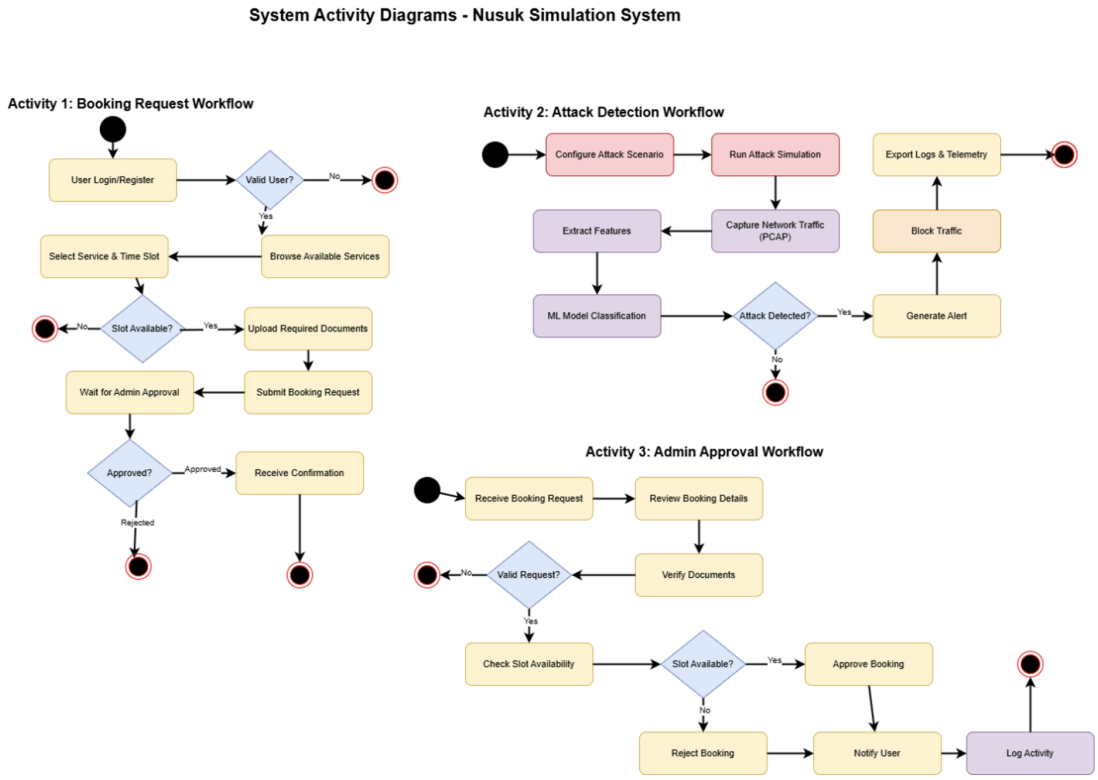


Figure 16 System Activity Diagram

This figure illustrates the **System Activity Diagram**, which represents the internal workflow of the system by showing the sequence of activities, decision points, and interactions between the user, the administrator, and the security mechanisms. The diagram is divided into three main workflows, each representing a distinct operational process within the system.

4 Booking Request Workflow

This workflow describes the steps followed by a user when submitting a booking request through the system. The process begins with user login or registration, followed by a validation step to verify the user's legitimacy. If the user is not valid, the process terminates. Upon successful validation, the user can browse available services and select a desired service and time slot. The system then checks the availability of the selected slot; if the slot is unavailable, the request is terminated. If the slot is available, the user uploads the required documents and submits the booking request. The request enters a pending state awaiting administrative approval. The workflow concludes either with the user receiving a booking confirmation upon approval or with rejection if the request is denied.

5 Attack Detection Workflow

This workflow illustrates the system's mechanism for detecting security attacks through simulation and intelligent analysis. The process starts with configuring an attack scenario, followed by running the attack simulation and capturing network traffic data in PCAP format. Relevant features are extracted from the captured traffic and passed to a machine learning classification model for analysis. The model determines whether an attack is present. If no attack is detected, the workflow ends. In the

event of an attack detection, the system generates a security alert, blocks the malicious traffic, and exports logs and telemetry data for further monitoring and analysis.

6 Administrative Approval Workflow

This workflow represents the administrator's role in processing booking requests. The process begins with receiving the booking request and reviewing its details. The administrator then verifies the submitted documents and evaluates the validity of the request. If the request is invalid, the process terminates. If valid, the administrator checks the availability of the requested time slot. When the slot is available, the booking is approved; otherwise, the booking is rejected. In both cases, the user is notified of the outcome, and the action is recorded in the system logs to ensure traceability and accountability.

The System Activity Diagram demonstrates the integration of user operations, administrative decision-making, and security monitoring within the system. It provides a clear behavioral view of how functional processes are executed, ensuring structured booking management, administrative control, and proactive security response, thereby enhancing the system's reliability and operational efficiency.

5.11 Front-End / User Interface Design

5.11.1 Landing Screen

This screen is the first interface the user interacts with.
It provides three entry options: Continue as a Guest, Login via Nafath, and the standard login button.
The design uses an Islamic-inspired background to reflect the identity of the Nusuk ecosystem.

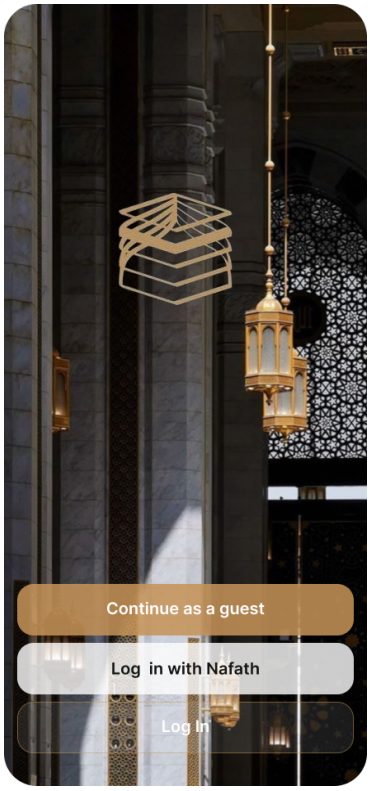


Figure 17 Landing Screen

5.11.2 Login Screen

The login screen allows users to authenticate using their National/Iqama ID and Password.
It offers a clean, minimal layout with a dark theme that improves readability.
A “Help logging in” link is included to support users facing access issues.

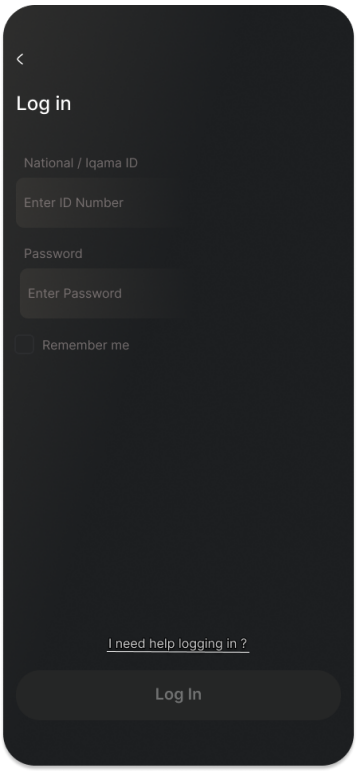


Figure 18 Login Screen

5.11.3 Services Home Screen

This is the main dashboard after logging in.
It provides quick access to major services such as **Flights Booking**, **Hotels Booking**, **Haramain Train**, **Nusuk Wallet**, and **My Reservations**.
The layout follows a simple grid design to ensure accessibility and ease of navigation.

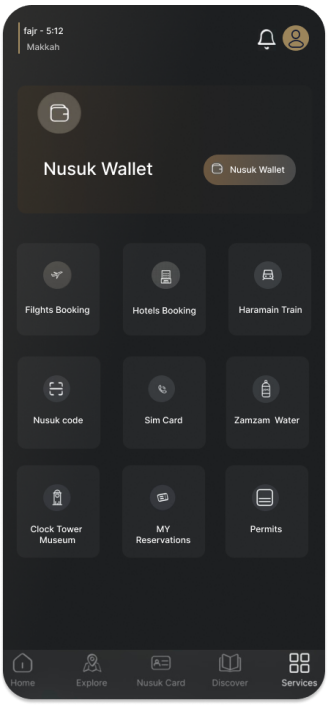


Figure 19 Services Home Screen

5.11.4 Dashboard

This dashboard gives admin an overview of their current booking status, including:

- Total Bookings
- Pending Requests
- Approved Services
- Documents Count

Users can also initiate a New Booking from this screen.
Recent bookings are displayed at the bottom with their date and status for quick reference.

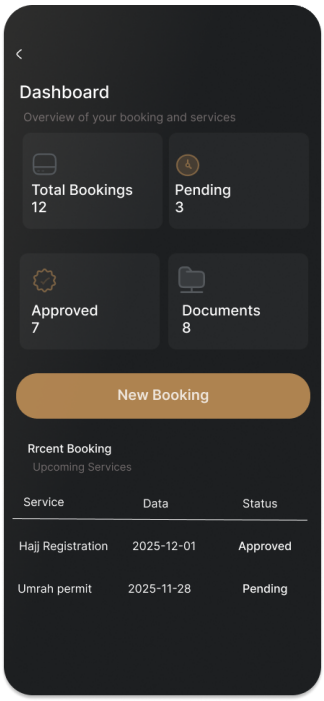


Figure 20 Dashboard

5.12 Database / Data Model Design

This section describes the database schema supporting the Nusuk Simulation System. The database uses a relational model with 10 normalized tables organized into functional areas.

5.12.1 Core Application Tables

Users: user_id (PK), username, email, password_hash, role (pilgrim/admin/developer), status. Relationships: One-to-Many with Bookings, Audit Logs, Attack Scenarios.

Services: service_id (PK), service_name, service_type, description, is_active.
Relationships: One-to-Many with Bookings and Time Slots.

TimeSlots: slot_id (PK), service_id (FK), slot_time, slot_date, available_capacity, total_capacity, status (available/full/closed).
Relationships: Many-to-One with Services, One-to-Many with Bookings.

Bookings: booking_id (PK), user_id (FK), service_id (FK), time_slot_id (FK), status (pending/approved/rejected), request_date, approval_date. Relationships: Many-to-One with Users, Services, Time Slots; One-to-Many with Documents.

Documents: document_id (PK), booking_id (FK), file_name, file_path, file_type, file_size, upload_date. Relationships: Many-to-One with Bookings.

AuditLogs: audit_id (PK), user_id (FK), action_type, entity_type, entity_id, old_values (JSON), new_values (JSON), timestamp.
Relationships: Many-to-One with Users.

5.12.2 Attack Simulation Tables

AttackScenarios: scenario_id (PK), scenario_name, attack_type, description, is_active, created_by (FK), parameters (JSON). Relationships: Many-to-One with Users, One-to-Many with Attack Logs.
AttackLogs: log_id (PK), scenario_id (FK), source_ip, timestamp, request_count, packet_count, flow_duration, pcap_file_path. Relationships: Many-to-One with Attack Scenarios, One-to-Many with Detection Results.

5.12.3 Detection and Alert Tables

Detection Results: detection_id (PK), log_id (FK), prediction (normal/attack), confidence_score, features_extracted (JSON), model_version, detection_timestamp. Relationships: Many-to-One with Attack Logs, One-to-Many with Alerts.

Alerts: alert_id (PK), detection_id (FK), severity (low/medium/high/critical), alert_type, status (open/investigating/resolved), created_at, resolved_at. Relationships: Many-to-One with Detection Results.

5.12.4 Design Features

- Primary keys (PK) are integer auto-increment; foreign keys (FK) maintain referential integrity
- ENUM types for status fields ensure data consistency
- JSON fields store flexible data (parameters, features, audit values)
- One-to-Many relationships use foreign keys on the "many" side
- Indexing on primary keys, foreign keys, and frequently queried fields

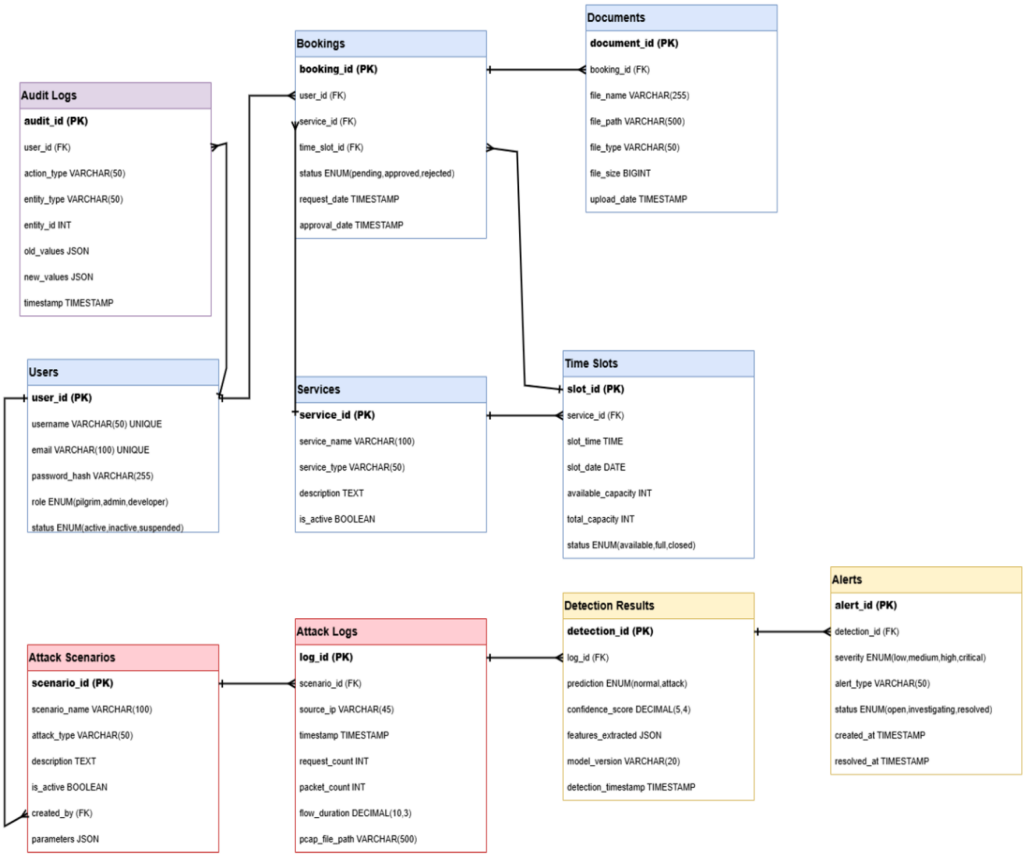


Figure 21 Database Schema Diagram

5.13 Algorithm / Process Design

This section describes the methodology and core algorithms used in the Nusuk Simulation System for detecting, filtering, and mitigating Denial-of-Service (DoS) attacks, as well as supporting booking operations within the application. The process covers dataset preparation, feature extraction, machine learning model training, real-time detection, and mitigation workflows.

5.13.1 Dataset Preparation

The CICDDoS2019 dataset was selected as the primary data source because it provides realistic and labeled network flows containing both benign traffic and various DoS/DDoS attack types. Multiple training files (e.g., *LDAP-training.parquet*, *DNS-training.parquet*) were merged into a unified training dataset, while all testing files were combined into a separate dataset to ensure evaluation on unseen traffic and prevent overfitting.

Each record in the dataset represents a network flow containing over 80 numerical features such as traffic rate, packet behavior, timing characteristics, and protocol flags. The label column distinguishes between benign and attack flows.

5.13.2 Data Preprocessing

Before training the model, several preprocessing steps were performed to prepare high-quality input:

1. **Removing Missing or Invalid Values**

Rows containing missing, infinite, or corrupted values were removed to ensure clean data.

2. **Feature Selection**

From the full feature set, the most relevant features for DoS detection were selected, including:

- Flow Duration
- Flow Packets per Second
- Flow Bytes per Second
- Packet Length Mean & Variance
- Inter-Arrival Time Features
- TCP Flags such as SYN, FIN, RST
- Request Rate & Packet Count patterns

These features capture anomalies in traffic behavior that commonly indicate DoS attacks.

3. **Normalization**

All selected features were normalized to a 0–1 numeric range to improve machine learning model stability and performance.

5.13.3 Attack Detection Algorithm

The system uses a machine learning–based detection module to identify DoS attacks in real time. **A Random Forest classifier was selected as the core detection model due to its robustness, high accuracy, and ability to handle high-dimensional traffic features commonly present in DoS/DDoS attack flows.**

Process Flow

1. **Traffic Capture:** Network traffic is logged with metadata (source IP, timestamps, request count, packet count, flow duration).
2. **Feature Extraction:** Features such as request rate, packet count, flow duration, and source IP frequency are computed.
3. **ML Prediction:** The trained model outputs:
    - Prediction: *normal* or *attack*
    - Confidence score (0.0–1.0)
4. **Alert Generation:** Alerts are created when the predicted class is “attack” and the confidence exceeds the threshold.

```
function detectAttack(trafficLog):
    features = extractFeatures(trafficLog)
    featureVector = normalize(features)
    prediction = mlModel.predict(featureVector)
    confidence = mlModel.getConfidence(featureVector)
    if prediction == "attack" and confidence > threshold:
        severity = calculateSeverity(confidence)
        alert = createAlert(prediction, confidence, severity)
        return alert
    return null
```

Figure 22 Attack Detection Algorithm

5.13.4 Feature Extraction Process

Feature extraction converts raw network traffic logs into structured numerical input for the model.

Extracted Features Include:

- Request Count
- Packet Count
- Flow Duration
- Request Rate (requests/sec)
- Source IP Frequency
- Pattern Deviations from Normal Baselines

Process

1. Input raw traffic log
2. Compute statistical and behavioral features
3. Normalize to 0–1 range
4. Output feature vector for ML prediction

5.13.5 Traffic Filtering and Mitigation Process

Once an attack is detected, the system activates mitigation mechanisms.

Process Flow

1. **Detection Trigger:** Raised by the attack detection algorithm
2. **IP Blocking:** Critical-severity attacks result in immediate blocking
3. **Rate Limiting:** Medium/high severity traffic is throttled
4. **Traffic Filtering:** Requests are filtered based on predefined security rules
5. **Logging:** All actions are logged for auditing and analysis

```
function filterTraffic(request, detectionResult):
    if detectionResult.prediction == "attack":
        if detectionResult.severity == "critical":
            blockIP(request.sourceIP)
            return BLOCKED
        else if detectionResult.severity == "high":
            if rateLimitExceeded(request.sourceIP):
                return BLOCKED
            applyRateLimit(request.sourceIP)
    return ALLOWED
```

Figure 23 Traffic Filtering and Response Based on Attack Severity

5.13.6 Booking Management Process

The system manages booking requests while maintaining time-slot availability and workflow approval.

Process Flow

1. User submits booking request
2. System validates availability and eligibility
3. Booking status initially set to **pending**
4. Admin reviews request
5. Request is **approved** or **rejected**
6. User is notified
7. Capacity is updated when approved

```
function processBooking(bookingRequest):
    if not validateAvailability(bookingRequest):
        return ERROR("Slot unavailable")
    booking = createBooking(bookingRequest)
    booking.status = "pending"
    notifyAdmin(booking)
    return booking
```

Figure 24 Booking Management Process

```
function approveBooking(bookingId):
    booking = getBooking(bookingId)
    booking.status = "approved"
    booking.approvalDate = now()
    updateCapacity(booking.timeSlotId, -1)
    notifyUser(booking)
```

Figure 25 Booking Approval Process

5.13.7 Attack Simulation Process

Attack simulations are used to test the detection pipeline and generate training data.

Process Flow

1. Developer selects attack scenario
2. Simulation executes in a controlled environment
3. Attack traffic is generated based on parameters
4. Traffic is captured into PCAP logs
5. Logs are stored with metadata
6. Extracted logs are used for model training or evaluation

```
function runAttackSimulation(scenarioId, parameters):
    scenario = getScenario(scenarioId)
    validateParameters(parameters)
    startCapture()
    executeAttack(scenario, parameters)
    attackLog = stopCapture()
    saveAttackLog(attackLog)
    return attackLog
```

Figure 26 Attack Simulation Process

5.13.8 Model Training Process

The ML model is trained to distinguish normal traffic from attack flows.

Process Flow

1. Collect normal and attack logs
2. Label the dataset
3. Perform feature extraction
4. Split into training (70%), validation (15%), test (15%)
5. Train the ML model
6. Validate performance using validation set
7. Test final accuracy using the test set
8. Deploy model if performance meets required accuracy threshold

5.14 Summary

This chapter presents the system design for the Nusuk Simulation System, covering architecture, use cases, data models, user interfaces, and core algorithms. This summary highlights the design decisions and their connection to implementation.

5.14.1 Design Overview

The system integrates two components:

1. **Booking Management System**: Handles pilgrim service bookings, approvals, and document management
2. **Attack Simulation and Detection System**: Provides DoS attack simulation, ML-based detection, and security research capabilities

The design follows a modular architecture with clear separation between application functionality and security research components.

5.14.2 Key Design Components

System Architecture (Section 5.1-5.3)

· Layered architecture with presentation, application, and data layers

· Separation of concerns between booking management and attack simulation

· Integration points for ML models, traffic capture, and detection services

Use Case Analysis (Section 5.4)

· Three actor types: Admin, Pilgrim/User, Developer/Research Team

· Unified use case specification for attack simulation and defense validation

· Clear workflows for booking management and security research activities

Data Flow and Interaction (Section 5.5-5.7)

· Data flow diagrams showing information flow between processes

· Sequence diagrams illustrating booking and attack detection workflows

· Activity diagrams for key business processes

Database Design (Section 5.12)

· 10 normalized tables organized by functional areas

· Core application tables: Users, Services, Time Slots, Bookings, Documents, Audit Logs

· Security tables: Attack Scenarios, Attack Logs, Detection Results, Alerts

· Relationships and constraints ensure data integrity

User Interface Design (Section 5.11)

· Clean, consistent interface with header, sidebar, and main content areas

· Three main screens: Login, Dashboard, Browse Services

· Multi-language support (Arabic/English) and dark mode

· Responsive design for different screen sizes

Algorithm Design (Section 5.13)

· ML-based attack detection algorithm with feature extraction

· Traffic filtering and mitigation processes

· Booking management workflow

· Attack simulation and model training processes

5.14.3 Design Decisions and Rationale

Modular Architecture: Enables independent development and maintenance of booking and security components. Normalized Database: 3NF design reduces redundancy and supports data integrity and scalability.ML-Based Detection: Real-time attack detection with confidence scoring supports automated response. Role-Based Access: Three user roles (Pilgrim, Admin, Developer) with appropriate permissions. Unified Attack Simulation: Single use case covering simulation, capture, detection, and model training.

5.14.4 Link to Implementation Stage

The design provides the foundation for implementation: **Database Implementation**

· Create 10 tables with defined schemas, relationships, and constraints

· Implement indexes on foreign keys and frequently queried fields

· Set up JSON fields for flexible data storage

Backend Implementation

· Develop REST APIs for booking management operations

· Implement attack simulation engine and traffic capture services

· Integrate ML model for real-time attack detection

· Build traffic filtering and mitigation components

Frontend Implementation

· Implement three main screens (Login, Dashboard, Browse Services) based on mockups

· Develop responsive layout with header, sidebar, and content areas

· Integrate multi-language support and dark mode toggle

· Create service cards, booking tables, and status indicators

ML Model Implementation

· Implement feature extraction pipeline from network traffic

· Deploy trained ML model for attack detection

· Build model training and retraining workflows

· Integrate confidence scoring and alert generation

Integration Points

· Connect frontend to backend APIs

· Integrate ML detection service with traffic capture

· Link attack simulation to detection and alerting systems

· Connect booking system with notification services

5.14.5 Implementation Priorities

The implementation process was organized into multiple development phases to ensure structured system design, gradual integration of components, and effective evaluation of the proposed DDoS detection framework.

Phase 1: Core database schema and basic booking management

Phase 2: User interface implementation and authentication

Phase 3: Attack simulation and traffic capture functionality

Phase 4: ML detection model integration and alerting

Phase 5: Advanced features (filtering, mitigation, model training)



Figure 27 Implementation Priorities

Figure ٢٥ Implementation priorities and phased development process of the proposed intelligent DDoS detection system.

7 Chapter 6

Implementation / Experimentation

7.1 Implementation Plan

The system development was carried out through several structured phases as follows:

7.1.1 Phase 1: Environment Setup

The development environment was prepared using the Python programming language. Several libraries and frameworks were installed to support machine learning, deep learning, and data preprocessing tasks, including:

- Scikit-learn for machine learning models
- TensorFlow / Keras for implementing the LSTM model
- Pandas and NumPy for data preprocessing and manipulation
- Matplotlib for visualization and result plotting

Additionally, the cybersecurity datasets used in this project were prepared, including:

- **CICDDoS2019**
- **UNSW-NB15**

7.1.2 Phase 2: Data Preparation

During this phase, both cybersecurity datasets — CICDDoS2019 and UNSW-NB15 — were preprocessed to ensure consistency, reliability, and suitability for machine learning and deep learning algorithms.

Several preprocessing operations were applied, including:

- Removing missing, invalid, and infinite values
- Standardizing feature structures across datasets
- Applying feature scaling and normalization using StandardScaler
- Aligning training and testing features to ensure consistency between experiments

For the CICDDoS2019 dataset, the labels were converted into binary classification classes:

- 0 = Normal Traffic
- 1 = Attack Traffic

Two evaluation approaches were implemented using CICDDoS2019:

- Per-attack evaluation
- Merged dataset evaluation

For the UNSW-NB15 dataset, only the merged dataset evaluation approach was used because the dataset already contains multiple attack categories within unified training and testing files.

Since this study focuses specifically on DoS attack detection, the labels in UNSW-NB15 were reorganized into three broader classes:

- DoS — traffic related to Denial-of-Service attacks
- Attack — all other malicious attack categories
- Non-Attack — normal benign traffic

Additionally, the datasets were divided into training and testing sets following an approximate 70/30 distribution.

For the LSTM model, the data was additionally reshaped into sequential format compatible with recurrent neural networks.

7.1.3 Phase 3: Model Development

Several machine learning and deep learning algorithms were implemented and evaluated, including:

- Support Vector Machine (SVM)
- Random Forest
- Logistic Regression
- Long Short-Term Memory (LSTM)

All models were trained and evaluated using the same preprocessing procedures and experimental settings to ensure a fair comparison between algorithms.

7.1.4 Phase 4: Evaluation and Comparison

The developed models were evaluated using standard classification metrics to measure their effectiveness in detecting DDoS attacks.

The evaluation process included the following metrics:

- **Accuracy**

  $$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

- **Precision**

  $$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

- **Recall**

  $$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$
- **F1-score**

  $$\text{F1} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Where:

- TP = True Positive
- TN = True Negative
- FP = False Positive
- FN = False Negative

To ensure a comprehensive and fair evaluation, two different experimentation scenarios were implemented using the CICDDoS2019 dataset.

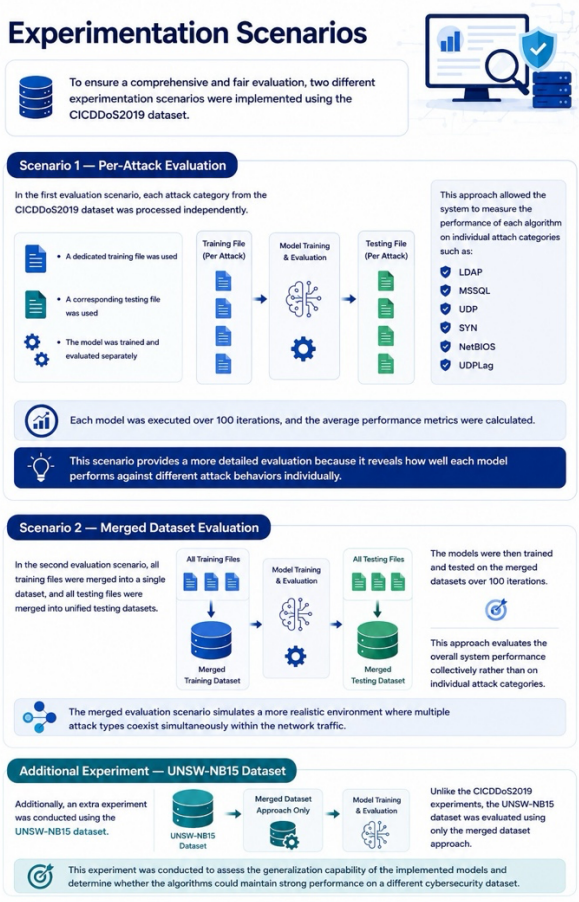


Figure 28. Experimental Evaluation Scenarios

7.1.4.1 *Scenario 1 — Per-Attack Evaluation*

In the first evaluation scenario, each attack category from the CICDDoS2019 dataset was processed independently.

For every attack type:

- A dedicated training file was used
- A corresponding testing file was used
- The model was trained and evaluated separately

This approach allowed the system to measure the performance of each algorithm on individual attack categories such as:

- LDAP
- MSSQL
- UDP
- SYN
- NetBIOS
- UDPLag

Each model was executed over 100 iterations, and the average performance metrics were calculated.

This scenario provides a more detailed evaluation because it reveals how well each model performs against different attack behaviors individually.

7.1.4.2 *Scenario 2 — Merged Dataset Evaluation*

In the second evaluation scenario, all training files were merged into a single dataset, and all testing files were merged into unified testing datasets.

The models were then trained and tested on the merged datasets over 100 iterations.

This approach evaluates the overall system performance collectively rather than on individual attack categories.

The merged evaluation scenario simulates a more realistic environment where multiple attack types coexist simultaneously within the network traffic.

Unlike the CICDDoS2019 experiments, the UNSW-NB15 dataset was evaluated using only the merged dataset approach.

This experiment was conducted to assess the generalization capability of the implemented models and determine whether the algorithms could maintain strong performance on a different cybersecurity dataset

7.1.4.3 Visualization of Model Performance

To better analyze the performance of the implemented machine learning and deep learning models, visual comparisons were generated for both evaluation scenarios.

The following figures illustrate the average accuracy results obtained from 100 runs for all implemented algorithms.

7.1.4.4 Per-Attack Evaluation Results

The following figure presents the average accuracy comparison of the implemented models when each attack type was evaluated separately.

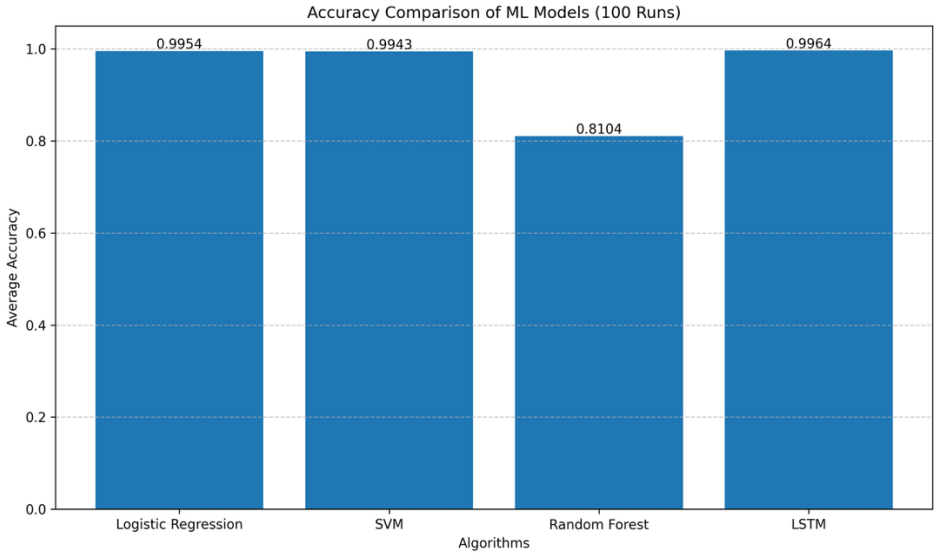


Figure 29 Accuracy comparison of machine learning models per file

Figure 29. Accuracy comparison of machine learning models using the per-attack evaluation approach over 100 runs.

The results indicate that the LSTM model achieved the highest accuracy, followed closely by Logistic Regression and SVM.

Random Forest achieved noticeably lower performance in this scenario because the model was trained independently on each attack dataset. Since some attack categories contained limited or highly imbalanced samples, the Random Forest model became more sensitive to variations between datasets, which negatively affected its average accuracy.

7.1.4.5 Merged Dataset Evaluation Results

The following figure presents the average accuracy comparison after merging all training datasets and testing datasets into unified datasets.

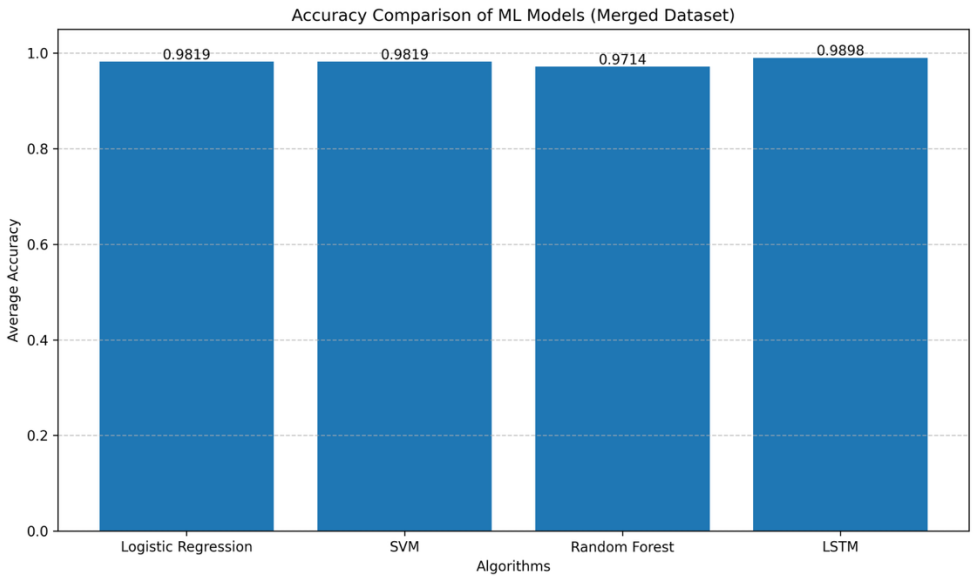


Figure 30 Accuracy comparison of machine learning models using the merged dataset evaluation approach over 100 runs on CICDDoS2019

The merged dataset evaluation produced more stable and balanced results across all models.

The performance of Random Forest improved significantly compared to the first scenario because the merged dataset provided:

- Larger training samples
- Better class distribution
- More diverse traffic patterns

Random Forest generally performs better with larger and more diverse datasets because it relies on multiple decision trees trained on random subsets of the data. When the datasets were merged, the model was able to generalize more effectively and reduce the impact of dataset-specific variations.

As a result, the accuracy gap between Random Forest and the other models became considerably smaller in the merged evaluation scenario.

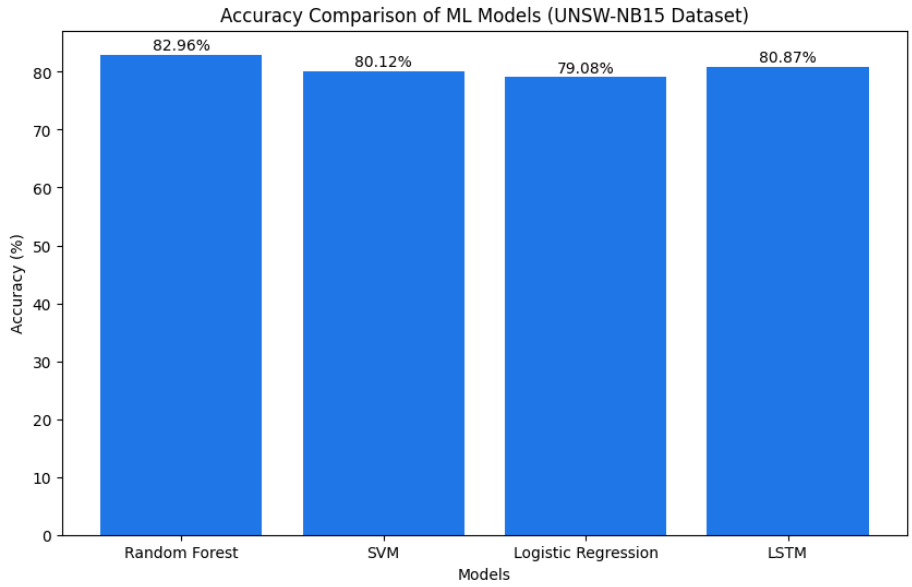


Figure 31 accuracy Comparison of ML Models on UNSW-NB15 Dataset

Figure 31 presents the accuracy comparison of the four evaluated machine learning models trained and tested on the UNSW-NB15 dataset. Random Forest achieved the highest accuracy of 82.96%, followed by LSTM at 80.87%, SVM at 80.12%, and Logistic Regression at 79.08%.

The performance gap between all models is relatively narrow, at approximately 3.88 percentage points indicating consistent and balanced performance across different model architectures on this dataset. Random Forest's leading performance on both CICDDoS2019 and UNSW-NB15 further confirms its suitability as the primary model for network attack detection in this study.

7.1.5 Phase 5: System Development

The system was implemented using Python to handle preprocessing, feature extraction, model training, prediction, and visualization tasks.

The implementation included:

- Data input and preprocessing
- Feature scaling and normalization
- Model training and evaluation
- Attack prediction
- Result generation and visualization

7.1.5.1 User Interface / Backend Development

7.1.5.1.1 Backend Development

The backend of the proposed system was implemented using the Python programming language. The backend environment handled all core functionalities related to data processing, machine learning execution, and performance evaluation.

The backend implementation included the following tasks:

- Loading cybersecurity datasets
- Data preprocessing and cleaning
- Feature scaling and normalization
- Training machine learning and deep learning models
- Executing prediction processes
- Calculating evaluation metrics
- Generating performance visualizations and comparison results

The implementation relied on several Python libraries, including:

- Scikit-learn
- TensorFlow / Keras
- Pandas
- NumPy
- Matplotlib

7.1.5.1.2 User Interface (UI) Development

The user interface was designed using Figma to create a realistic prototype representing the proposed cyberattack detection system and user interaction workflow.

The prototype was designed to simulate how the system would appear and operate in a real-world environment.

The interface included several components, such as:

- Login screen
- Services interface simulating the Nusuk application
- Security monitoring dashboard
- Attack alert interfaces
- Detection result visualization

The interface was designed with a simple and user-friendly layout to ensure ease of use and clear presentation of attack detection results.

Although the interface was not fully integrated with the backend implementation, it provides a realistic visualization of how the proposed system could operate in practice.

7.1.5.1.3 User Interface Prototype Visualization

The following figures illustrate the designed user interface prototype created using Figma. These screens demonstrate how users interact with the system and how attack detection results are presented to the administrator in real time

1. Login Interface

This screen allows the administrator to securely access the system. It includes:

- Admin Username / ID input field
- Password input field
- “Remember Me” option
- Log in button
- Option to log in via **Nafath**

The interface is designed with a clean and professional layout, featuring a background that reflects the official nature of the application, enhancing user trust and security.

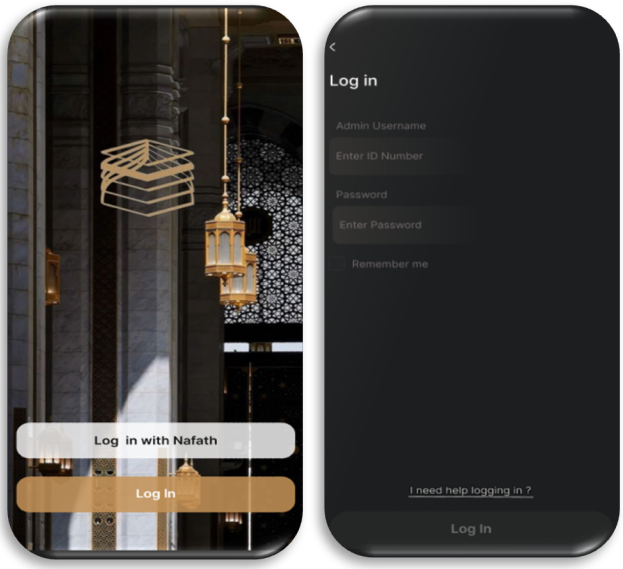


Figure 30 Login Interface

2. Security Dashboard

This interface represents the main monitoring dashboard for the administrator and provides real-time system monitoring information.

The dashboard includes:

- System Status
- Active Users
- Requests per Second
- Traffic Overview chart
- Detected Attacks section
- Detection statistics and alerts

The dashboard provides administrators with a comprehensive overview of network activity and helps support rapid decision-making during cyberattacks.

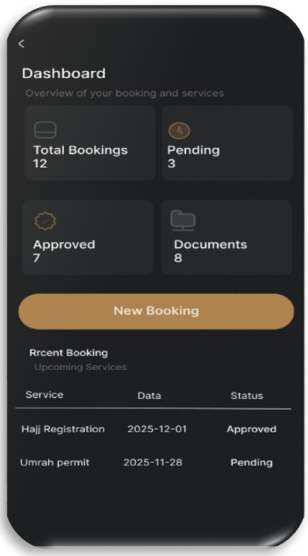


Figure 32 Security Dashboard

3. Security Alert Interface

This interface appears when suspicious activity or a cyberattack is detected.

The alert interface includes:

- Attack type (e.g., DDoS Attack)
- Source IP address
- Attack timestamp
- Action taken by the system
- “View Logs” button
- “Dismiss” button

This interface allows administrators to quickly monitor detected threats and review system responses.

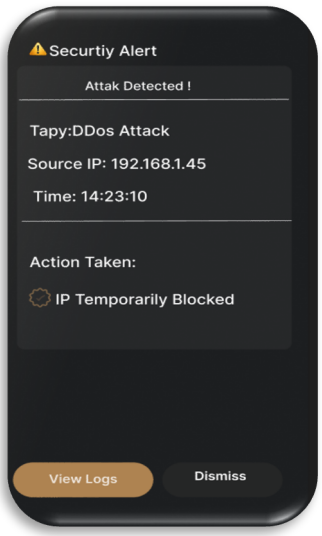


Figure 33 Security Alert Interface

7.2 Deployment or Execution

The proposed system was implemented within a controlled and secure experimental environment.

7.2.1 Operating Characteristics

- The system operates in a local environment
- The implementation was not deployed in a real production environment
- The experiments relied on pre-prepared cybersecurity datasets
- The execution environment was isolated to ensure safe experimentation

7.2.2 Execution Steps

The system execution process included:



Figure 34 Execution Steps

This controlled experimental setup ensured that all experiments were conducted safely without affecting real-world systems or network environments.

7.3 Summary

This chapter presented the implementation and experimentation phase of the proposed cyberattack detection system.

Several machine learning and deep learning models were implemented and evaluated using two cybersecurity datasets: CICDDoS2019 and UNSW-NB15.

To ensure a comprehensive evaluation, two experimental approaches were applied on the CICDDoS2019 dataset:

- Per-attack evaluation
- Merged dataset evaluation

In contrast, the UNSW-NB15 dataset was evaluated using only the merged dataset approach due to its unified structure and multiple embedded attack categories.

The experiments demonstrated that all implemented models achieved strong detection performance across different evaluation scenarios. The results also showed that model performance varied depending on the dataset structure and evaluation methodology.

Random Forest achieved the most stable and consistent overall performance, particularly in the merged dataset evaluation and UNSW-NB15 experiments, confirming its suitability for network attack detection within this study. The merged dataset approach also provided more balanced and realistic evaluation conditions by combining multiple attack patterns into unified datasets.

The implementation was conducted within a controlled experimental environment that enabled accurate, stable, and secure performance analysis.

The next chapter presents the experimental results and provides a detailed discussion and comparative analysis of the implemented models.

8 Chapter 7

Results and Evaluation

8.1 Results Presentation

The performance of the machine learning models used in the system was evaluated to determine the most efficient model for detecting cyber attacks. The models included:

- Support Vector Machine (SVM)
- Random Forest
- Logistic Regression
- Long Short-Term Memory (LSTM)

All models were evaluated using two datasets: CICDDoS2019 as the primary dataset, and UNSW-NB15 as a secondary dataset for cross-dataset evaluation. Both datasets used independent train/test splits to ensure a fair and consistent comparison across all implemented models.

8.1.1 Evaluation Metrics

Four main metrics were used:

- **Accuracy**: Measures the proportion of correct predictions out of all predictions.
- **Precision**: Measures the accuracy of positive predictions (attacks).
- **Recall**: Measures the model's ability to detect all actual attacks.
- **F1-score**: Represents the balance between precision and recall.

8.1.2 Performance Evaluation

To evaluate the performance of the implemented machine learning models, several standard classification metrics were utilized:

- **Accuracy:** Indicates the percentage of correctly classified instances among all predictions.
- **Precision:** Reflects how many of the predicted attack instances are actually true attacks.
- **Recall (Detection Rate):** Measures the model's ability to correctly identify all actual attack instances.
- **F1-score:** Provides a balanced measure that combines both precision and recall into a single metric.
- **Confusion Matrix:** Offers a detailed breakdown of prediction outcomes by comparing actual and predicted classes, highlighting true positives, true negatives, false positives, and false negatives.

8.1.2.1 Model Performance Comparison on CICDDoS2019 Dataset (merged)

Model	Accuracy	Precision	Recall	F1-score
Random Forest	97.2%	99.9%	96.8%	98.9%
SVM	98.1%	99.6%	97.3%	98.4%
Logistic Regression	98.8%	96.2%	95.7%	95.9%
LSTM	98.5%	90.8%	98.6%	99.2%

Table 31Performance Comparison on CICDDoS2019

8.1.2.2 Model Performance on UNSW-NB15 Dataset

Model	Accuracy	Precision	Recall	F1-score
Random Forest	82.96%	81.07%	82.96%	80.75%
SVM	80.12%	79.74%	80.12%	77.34%
Logistic Regression	79.08%	78.09%	79.08%	77.63%
LSTM	80.87%	80.23%	80.87%	79.67%

Table 32 Performance on UNSW-NB15 Dataset

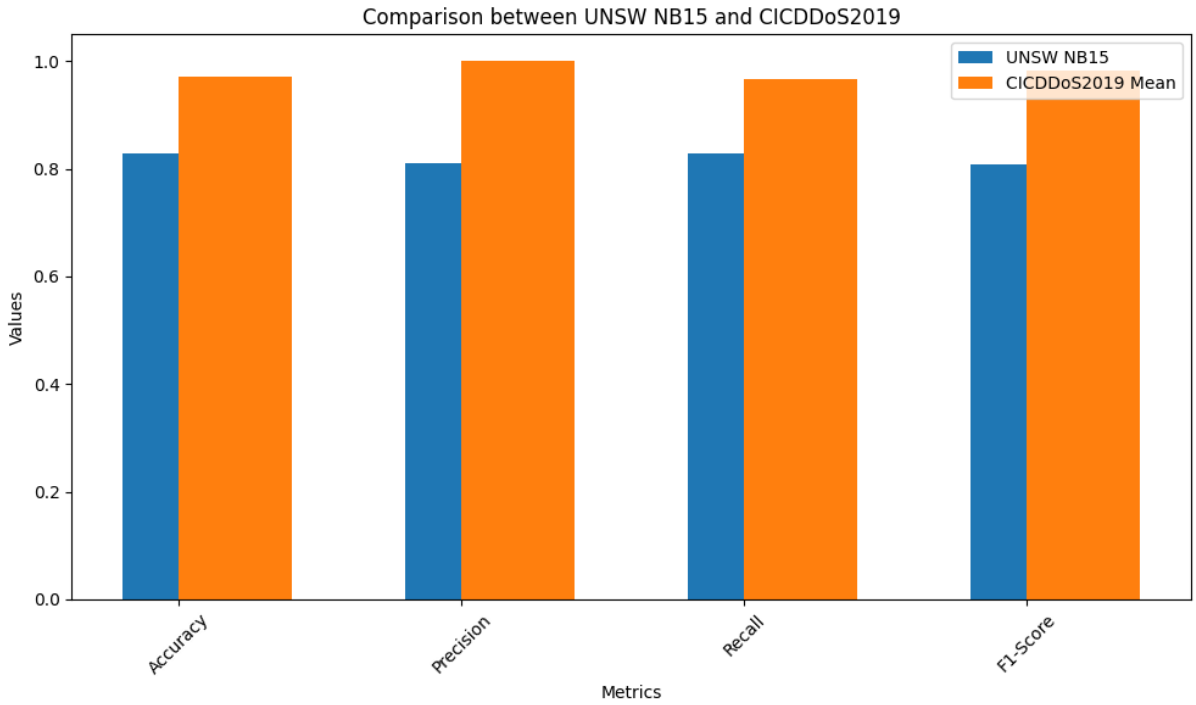


Figure 35 Comparison between UNSW NB15 and CICDDos2019 on Random Forest

Figure 35 presents a direct comparison of the Random Forest model performance across both datasets CICDDoS2019 and UNSW-NB15 evaluated on four metrics: Accuracy, Precision, Recall, and F1-Score.

The model achieved consistently higher performance on CICDDoS2019 across all metrics, with Accuracy of 97.2%, Precision of 99.1%, Recall of 98.8%, and F1-Score of 98.9%. On UNSW-NB15, the model recorded Accuracy of 82.96%, Precision of 81.07%, Recall of 82.96%, and F1-Score of 80.75%.

The performance gap between the two datasets is expected and can be attributed to the inherent differences in network traffic characteristics, attack types, and class distributions between CICDDoS2019 and UNSW-NB15. CICDDoS2019 focuses specifically on DDoS attack patterns, while UNSW-NB15 encompasses a broader range of network anomalies, making classification a more challenging task.

Despite this gap, Random Forest maintained solid performance on both datasets, confirming its robustness and consistency as the best-performing model in this study

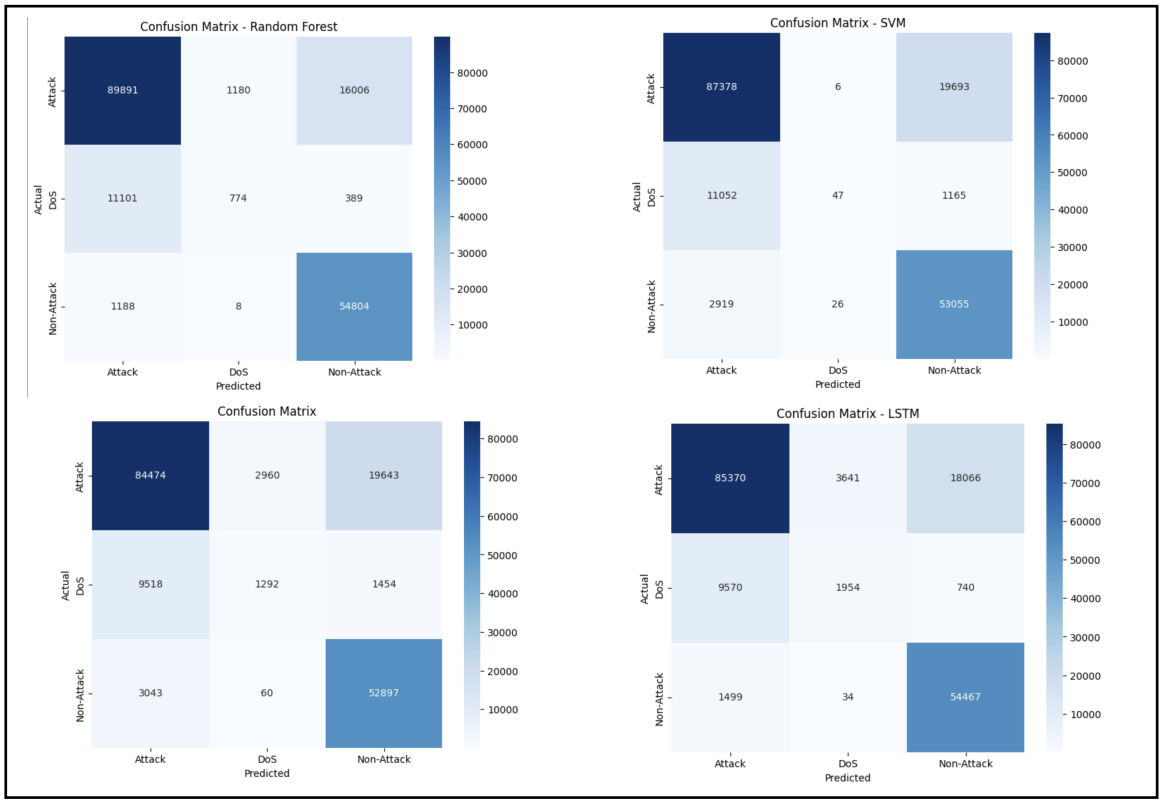


Figure 35 confusion matrices on UNSW-NB15

Analysis of the confusion matrices on UNSW-NB15 reveals that all four models struggle most with the DoS class, which is the least represented category in the dataset. This misclassification pattern is consistent across models and indicates a class imbalance challenge rather than a model-specific limitation. Random Forest achieved the best overall diagonal performance, correctly classifying the majority of Attack and Non-Attack instances, while DoS detection remains an area for improvement across all evaluated models.

8.2 Usability Testing / User Feedback

Usability testing was conducted to evaluate the ease of use and user experience of the system.

8.2.1 Key Findings:

- The dashboard is clear and easy to navigate
- Alerts are understandable and informative
- Users can interact with the system without technical difficulties

8.2.2 Suggested Improvements:

- Add more visualizations such as graphs and charts
- Provide more customization options for alerts

Overall, user feedback was positive, indicating good usability.

8.3 Comparative Analysis

The system was compared with existing methods and approaches reported in the literature.

8.3.1 Comparison with Related Work

The system was compared with existing methods and approaches reported in the literature. In the study by Moustafa and Slay [24], anomaly detection techniques were used for cyberattack detection, achieving an accuracy of 93.5%. In comparison, our system, utilizing Random Forest on the UNSW-NB15 dataset, achieved an accuracy of 82.96%. This difference in performance highlights the challenges that arise from using different datasets. However, this comparison demonstrates the effectiveness of the Random Forest model across diverse data environments, reflecting its strength in handling network traffic data.

8.3.2 Key Observations:

• The experimental results demonstrated that all implemented models achieved high performance in detecting DDoS attacks across different evaluation scenarios.

• In the CICDDoS2019 merged dataset evaluation, the LSTM model achieved the highest overall accuracy, while SVM and Logistic Regression also showed highly stable and competitive performance.

• Random Forest showed lower performance in the per-attack evaluation scenario but improved significantly after applying the merged dataset approach. The larger and more diverse merged dataset enabled the model to generalize more effectively and reduced the impact of dataset-specific variations.

• The merged dataset evaluation produced more balanced and stable results across all implemented models, indicating that combining multiple attack categories improves model robustness and overall detection capability.

• In the UNSW-NB15 generalization experiment, Random Forest achieved the highest accuracy among all evaluated models, demonstrating strong generalization capability on a completely different cybersecurity dataset.

• The results also showed that the performance gap between the evaluated models became relatively small in the UNSW-NB15 experiment, indicating stable behavior across different model architectures and dataset distributions.

• Overall, the proposed intelligent DDoS detection framework demonstrated effective and reliable performance under multiple experimental conditions.

8.3.3 Discussion of Findings

The experimental results demonstrate that model selection has a significant impact on the overall performance of DDoS attack detection systems.

Different models showed different strengths depending on the evaluation scenario and dataset structure. The LSTM model achieved the highest performance in the merged CICDDoS2019 evaluation scenario, indicating its strong capability in learning complex traffic patterns from larger datasets.

Random Forest demonstrated noticeable improvement when evaluated using the merged dataset approach and achieved the best performance on the UNSW-NB15 dataset. This suggests that Random Forest performs effectively when trained on larger and more diverse datasets and maintains strong generalization capability across different traffic distributions.

Traditional machine learning models such as SVM and Logistic Regression also achieved highly competitive and stable results throughout the experiments, confirming their effectiveness for structured network traffic classification tasks.

The experiments further showed that merged dataset evaluation provides more stable and balanced performance compared to evaluating each attack category independently. Combining multiple attack types allows the models to learn broader traffic characteristics and reduces sensitivity to individual dataset variations.

These findings highlight the importance of selecting suitable machine learning models and evaluation strategies based on dataset characteristics, attack diversity, and generalization requirements in cybersecurity systems.

8.4 Challenges Encountered and Lessons Learned

Challenges:

- Handling large and complex datasets
- Differences between datasets
- Hyperparameter tuning for each model

Solutions:

- Applied data preprocessing and normalization
- Used multiple datasets to improve generalization
- Performed model tuning to optimize performance

Lessons Learned:

- Data quality significantly affects results
- Simpler models can sometimes outperform complex ones
- Testing on multiple datasets is essential for reliable evaluation

Limitations

The study has several limitations:

- Limited dataset size compared to real-world scenarios
- The system was not tested in a real-time deployment environment
- Only a limited number of models were evaluated

These limitations may affect the generalizability of the results.

8.5 Summary

This section presented a comprehensive evaluation of the proposed system, including functional testing, performance evaluation, usability testing, and comparative analysis.

The results demonstrated that:

• The system operates correctly and reliably under different experimental conditions.
• The implemented machine learning and deep learning models achieved high performance in detecting DDoS attacks.
• The LSTM model achieved the highest performance in the merged CICDDoS2019 evaluation scenario, while Random Forest demonstrated strong generalization capability on the UNSW-NB15 dataset.

• The system maintained stable and competitive performance across different datasets and evaluation scenarios.

Overall, the proposed intelligent DDoS detection system shows strong potential for efficient, reliable, and scalable cyberattack detection.

9 Chapter 8

Conclusion and Future Work

9.1 Summary of Work

This project aimed to develop an intelligent machine learning-based system for detecting cyberattacks, particularly Distributed Denial of Service (DDoS) attacks, through the analysis of network traffic and classification of traffic flows as either normal or malicious.

Several machine learning and deep learning algorithms were implemented and evaluated, including:

• Support Vector Machine (SVM)
• Random Forest
• Logistic Regression
• Long Short-Term Memory (LSTM)

The implemented models were evaluated using two cybersecurity datasets:

• CICDDoS2019
• UNSW-NB15

To ensure a comprehensive and fair evaluation, two experimentation scenarios were applied using the CICDDoS2019 dataset:

• Per-Attack Evaluation
• Merged Dataset Evaluation

In addition, a generalization experiment was conducted using the UNSW-NB15 dataset to evaluate the robustness of the implemented models on a completely different network traffic distribution.

The experimental results demonstrated that all implemented models achieved high detection performance across different evaluation scenarios. The LSTM model achieved the highest overall performance in the merged dataset evaluation scenario, while Random Forest demonstrated strong generalization capability and stable performance on the UNSW-NB15 dataset.

Overall, the proposed system demonstrated the ability to:

• Accurately detect cyberattacks
• Distinguish between normal and malicious traffic
• Maintain stable performance across different datasets
• Generalize effectively under different experimental conditions

These results indicate that machine learning and deep learning techniques can provide an effective foundation for intelligent cybersecurity systems.

9.2 Key Findings

The key findings of this project can be summarized as follows:

• The implemented machine learning and deep learning models achieved high performance in detecting DDoS attacks across different datasets and evaluation scenarios.

• The LSTM model achieved the highest overall accuracy in the merged CICDDoS2019 evaluation scenario, demonstrating strong capability in learning complex traffic behavior patterns.

• Random Forest showed significant improvement when evaluated using the merged dataset approach and achieved the highest accuracy on the UNSW-NB15 dataset, indicating strong generalization capability across different traffic distributions.

• SVM and Logistic Regression also achieved stable and competitive results, confirming their effectiveness for structured network traffic classification tasks.

• The merged dataset evaluation approach produced more balanced and reliable results compared to evaluating each attack category independently.

• The experiments demonstrated that using multiple datasets improves model robustness and helps evaluate system performance under different cybersecurity conditions.

9.3 Future Work

Several improvements can be implemented in future versions of the proposed system:

8.3.1 Improving Model Performance

The system performance can be further enhanced by:

• Using more advanced algorithms such as XGBoost or Transformer-based models
• Applying advanced feature engineering and feature selection techniques
• Performing hyperparameter optimization for all implemented models

9.3.1 Real-Time Implementation

The proposed system can be extended to operate in real-time environments by:

• Monitoring live network traffic
• Detecting attacks as they occur
• Integrating automated alert and response mechanisms

9.3.2 Expanding Attack Detection

Future work may include extending the system to detect additional cyberattack categories such as:

• SQL Injection
• Phishing attacks
• Malware attacks
• Insider threats

9.3.3 Enhancing the User Interface

The designed Figma prototype can be further improved by:

• Converting the prototype into a fully functional application
• Integrating the frontend with the machine learning backend
• Providing interactive dashboards and visualization tools

9.3.4 Advanced AI Integration

Future research may explore the integration of:

• Advanced Deep Learning techniques
• Reinforcement Learning
• Hybrid Intrusion Detection Systems (IDS)
• Explainable AI (XAI) techniques for cybersecurity analysis

9.4 Final Conclusion

This project demonstrated how machine learning and deep learning techniques can effectively contribute to cyberattack detection and network traffic analysis.

The experimental results confirmed that evaluating models using multiple datasets and multiple experimentation scenarios provides more reliable and realistic performance analysis.

Overall, the proposed intelligent DDoS detection framework provides a strong foundation for developing scalable and intelligent cybersecurity systems capable of operating in more realistic and dynamic environments.

References

[1] Cloudflare, "What is a DDoS attack?," 2025. [Online]. Available: https://www.cloudflare.com/.

[2] I. C. S. &. Tutorials, "A Survey of Defense Mechanisms Against Distributed Denial of Service (DDoS) Flooding Attacks," 2013.

[3] SOCRadar, "Cyber Threats Targeting the Hajj Season," 2024. [Online]. Available: https://socradar.io/blog/cyber-threats-targeting-the-hajj-season/.

[4] CISA, "Recognize and Report Phishing," 2025. [Online]. Available: https://www.cisa.gov/.

[5] O. Foundation, "OWASP API Security Top 10," 2023. [Online]. Available: https://owasp.org/www-project-api-security/.

[6] O. –. W. S. T. Guide, "Web Security Testing Guide," 2023. [Online]. Available: https://owasp.org/www-project-web-security-testing-guide/.

[7] OWASP, "SQL Injection," 2025. [Online]. Available: https://owasp.org/www-community/attacks/SQL_Injection?utm_source=chatgpt.com.

[8] Kaspersky, "Data Breaches & Privacy Risks," [Online]. Available: https://www.kaspersky.com/resource-center/definitions/data-breach?utm_source=chatgpt.com.

[9] HackRead, "Hajj Pilgrimage Hit by Extensive Phishing and Data Theft Scams," 2024. [Online]. Available: https://hackread.com/hajj-pilgrimage-phishing-data-theft-scams/?utm_source=chatgpt.com.

[10] T. S. UK, 2023. [Online]. Available: https://www.tradingstandards.uk/news-policy-campaigns/blogs/hajj-pilgrimage-and-being-wary-of-scams/?utm_source=chatgpt.com.

[11] Kaspersky, "What is a man-in-the-middle (MitM) attack?," 2025. [Online]. Available: https://encyclopedia.kaspersky.com/glossary/man-in-the-middle-attack/?utm_source=chatgpt.com.

[12] Snyk, "Man-in-the-Middle Attacks and How to Prevent Them," [Online]. Available: https://snyk.io/articles/between-you-and-the-data-defending-against-man-in-the-middle-attacks/?utm_source=chatgpt.com.

[13] O. Foundation, "XSS Cheat Sheet," 2023. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html?utm_source=chatgpt.com.

[14] A. TOSEM, "ML Models for Predicting Software Vulnerabilities," 2022. [Online]. Available: https://dl.acm.org.

[15] U. S. Symposium, "Detecting Logic Abuse in APIs," 2021. [Online]. Available: https://www.usenix.org.

[16] I. C. S. &. Tutorials, "A Survey of ML-based DDoS Detection Techniques," 2021. [Online]. Available: https://ieeexplore.ieee.org/document/9350318.

[17] B. M. Kanber, N. F. Noaman, A. M. H. Saeed and M. Malas, "DDoS Attacks Detection in the Application Layer Using Three Level Machine Learning Classification Architecture," 2022. [Online]. Available: https://doi.org/10.5815/ijcnis.2022.03.03.

[18] I. S. &. Privacy, "Understanding User Susceptibility to Phishing," 2020. [Online]. Available: https://ieeexplore.ieee.org.

[19] Springer, "Anomaly-Based Detection of DB Exfiltration Activities," 2020. [Online]. Available: https://link.springer.com.

[20] A. CCS, "Detecting TLS Downgrade & Certificate Spoofing," 2019. [Online]. Available: https://dl.acm.org.

[21] I. S. &. Privacy, "Empirical Study of Targeted Social Engineering," 2021. [Online]. Available: https://ieeexplore.ieee.org.

[22] U. o. N. B. Canadian Institute for Cybersecurity (CIC), "CIC-DDoS 2019 Dataset," 2019. [Online]. Available: https://www.kaggle.com/datasets/dhoogla/cicddos2019.

[23] N. B. D. K. &. K. J. K. Hoque, "Botnet in DDoS attacks: Trends and challenges," [Online]. Available: https://doi.org/10.1109/COMST.2015.2468553.

[24] N. &. S. J. Moustafa, "The evaluation of network anomaly detection systems: Statistical analysis of the UNSW-NB15 dataset and applications," 2016. [Online].

[25] B. &. Guven, "A survey of data mining and machine learning methods for cyber security intrusion detection," [Online]. Available: https://doi.org/10.1109/COMST.2015.2494502.

[26] Kaggle, "UNSW_NB15 Dataset," Kaggle.

[27] U. o. N. S. Wales, UNSW-NB15, UNSW Research, 2021.

[28] Project Management Institute, A Guide to the Project Management Body of Knowledge (PMBOK® Guide)A Guide to the Project Management Body of Knowledge (PMBOKGuide)A Guide to the Project Management Body of Knowledge.

[29] StormWall, "Cyberstorm in MENA: DDoS Attack Report for Q2 2025," 1 1 2025. [Online]. Available: https://stormwall.network/resources/blog/q2-2025-ddos-report-mena.

[30] D. Reading, "Articles on DDoS attacks targeting," 2025. [Online]. Available: https://www.darkreading.com/.

[31] A. News, "Saudi Arabia issues warning about Hajj, Umrah online scams, fake websites," 2023. [Online]. Available: https://www.arabnews.com/.

[32] Kaspersky, " Articles on data breaches and privacy risks," 2025. [Online]. Available: https://www.kaspersky.com/.